



**ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN**

Football Analysis System

BÁO CÁO ĐỒ ÁN

Nhận dạng CS338.Q22

Giảng viên hướng dẫn: Dương Việt Hằng

Sinh viên thực hiện:

Phan Tiến Đạt - 24520288

Vũ Quang Dũng - 22520294

Ngày 20 tháng 6 năm 2026

Mục lục

Tóm Tắt Đồ Án	4
1 Chương 1: Giới Thiệu	4
1.1 Bối cảnh và vấn đề đặt ra	4
1.2 Mục tiêu đề tài	6
1.3 Phạm vi	7
2 Chương 2: Cơ Sở Lý Thuyết	8
2.1 Phát hiện đối tượng: YOLOv11s	8
2.1.1 Kiến trúc tổng quát của YOLO	8
2.1.2 Mô hình YOLOv11s	8
2.2 Theo dõi đa đối tượng: ByteTrack	9
2.2.1 Bài toán MOT và Bộ lọc Kalman	9
2.2.2 Thuật toán Hungarian và Cơ chế Khớp kép (Double Matching)	10
2.3 Tái định danh người: OSNet	11
2.3.1 Bài toán Person Re-Identification (ReID)	11
2.3.2 OSNet – Omni-Scale Feature Learning	12
2.3.3 Hàm mất mát huấn luyện	13
2.3.4 Độ tương đồng Cosine và ngưỡng quyết định	13
2.4 Biến đổi phối cảnh: Homography	13
2.5 Ước lượng chuyển động camera: Optical Flow	14
2.5.1 Giả định độ sáng bất biến	14
2.5.2 Giải bằng phương pháp Lucas–Kanade	14
2.5.3 Phiên bản kim tự tháp (Pyramidal LK)	15
2.5.4 Ước lượng và bù trừ chuyển động	16
2.6 Phân cụm màu áo: Thuật toán K-Means	16
2.6.1 Bài toán phân cụm	16
2.6.2 Quy trình lặp	16
2.6.3 Trích xuất màu áo cầu thủ	17
2.7 Ước lượng tốc độ và quãng đường	17
2.7.1 Quãng đường di chuyển	18
2.7.2 Tốc độ tức thời	18
2.8 Gán bóng và tính kiểm soát bóng	18
2.8.1 Gán bóng cho cầu thủ	18
2.8.2 Tỷ lệ kiểm soát bóng	18

3	Chương 3: Phương Pháp Thực Hiện	19
3.1	Sơ đồ kiến trúc tổng thể	19
3.2	Mô tả bài toán	20
3.2.1	Đầu vào (Input)	20
3.2.2	Đầu ra (Output)	21
3.2.3	Minh họa	22
3.3	Dataset và huấn luyện mô hình	22
3.3.1	Hai mô hình cần huấn luyện	22
3.3.2	Thu thập và gán nhãn dữ liệu	24
3.3.3	Huấn luyện mô hình	25
3.4	Module Tracking (<code>tracker.py</code>)	26
3.4.1	Phát hiện theo lô (batch)	26
3.4.2	Gộp lớp goalkeeper vào player	26
3.4.3	Tích hợp ByteTrack	26
3.4.4	Cấu trúc dữ liệu <code>tracks</code>	27
3.4.5	Nội suy vị trí bóng	28
3.4.6	Thêm điểm vị trí đại diện	28
3.4.7	Cơ chế stub	28
3.5	Ước lượng chuyển động camera (<code>camera_movement_estimator.py</code>)	29
3.5.1	Trích đặc trưng từ vùng nền tĩnh	29
3.5.2	Tính độ dịch chuyển bằng Lucas–Kanade	30
3.5.3	Hiệu chỉnh vị trí đối tượng	30
3.6	Biến đổi phối cảnh — Homography động (<code>view_transformer.py</code>)	31
3.6.1	Cấu hình sân chuẩn	31
3.6.2	Ước lượng ma trận H động theo từng khung hình	31
3.6.3	Cơ chế dự phòng (fallback)	32
3.6.4	Áp dụng phép biến đổi	33
3.6.5	Cơ chế stub cho Homography	33
3.7	Gán đội theo màu áo (<code>team_assign.py</code>)	34
3.7.1	Trích xuất màu áo cầu thủ	34
3.7.2	Phân hai đội	34
3.7.3	Gán đội và lưu cache	35
3.8	Module ReID — 5 tầng (<code>reid.py</code>)	35
3.8.1	Trích đặc trưng OSNet	35
3.8.2	Kiến trúc gallery	36
3.8.3	Năm tầng so khớp	36
3.8.4	Cập nhật gallery	37
3.8.5	Hợp nhất offline, ánh xạ ID và stub	37
3.9	Tốc độ và quãng đường (<code>speed_and_distance_estimator.py</code>)	38

3.10	Kiểm soát bóng (player_ball_assign.py và logic trong main.py)	39
3.11	Trực quan hóa: Tactical Map & Pitch Renderer	40
3.11.1	Render sân 2D (PitchRenderer)	40
3.11.2	Tactical Map (TacticalMap)	40
3.12	Tổng hợp thống kê và sinh báo cáo (stats_aggregator, nlp_report)	41
3.12.1	Tổng hợp thống kê (stats_aggregator)	41
3.12.2	Sinh báo cáo tự động bằng LLM (nlp_report)	42
4	Chương 4: Thực Nghiệm Và Đánh Giá	43
4.1	Mô tả tập dữ liệu	43
4.1.1	Thu thập dữ liệu	43
4.1.2	Thống kê tập dữ liệu	43
4.1.3	Tiền xử lý	44
4.2	Dữ liệu thực nghiệm	44
4.3	Độ đo đánh giá	44
4.3.1	Độ chính xác phát hiện	44
4.3.2	Confusion Matrix	44
4.3.3	Độ ổn định Tracking và ReID	45
4.4	Kết quả thực nghiệm Detection	45
4.4.1	Quá trình huấn luyện	45
4.4.2	Kết quả tổng hợp	46
4.4.3	Phân tích các đường cong	46
4.4.4	Phân tích Confusion Matrix	49
4.5	Đánh giá Tracking và Re-Identification	50
4.6	Đánh giá Homography, Tốc độ và Quãng đường	51
4.7	Đánh giá Báo cáo chiến thuật	51
5	Chương 5: Kết Luận Và Hướng Cải Tiến	52
5.1	Điểm mạnh	52
5.2	Hạn chế	53
5.3	Hướng cải tiến	53
5.3.1	Cải thiện mô hình	53
5.3.2	Mở rộng chức năng	54
5.3.3	Tối ưu triển khai	54
5.4	Kết Luận	54
	Tài Liệu Tham Khảo	55

Phân công công việc

Tên	Công việc thực hiện	Độ hoàn thành
Phan Tiến Đạt	Xử lý dữ liệu và tiền xử lý dữ liệu. Huấn luyện mô hình phát hiện đối tượng (player, goalkeeper, referee, ball). mô hình phát hiện điểm mốc sâu (keypoint). Xây dựng các module ByteTrack, ReID và Homography. Xử lý logic phân đội theo màu áo, ước tính tốc độ, quãng đường di chuyển và kiểm soát bóng. Trực quan hóa bản đồ chiến thuật 2D.	100%
Vũ Quang Dũng	Xây dựng module sinh sơ đồ chiến thuật. Xây dựng prompt và tích hợp mô hình ngôn ngữ lớn (LLM) để sinh phân tích chiến thuật. Xuất báo cáo chiến thuật định dạng PDF.	100%

Bảng 1: Phân công công việc giữa các thành viên

Tóm Tắt Đề Án

Đề án xây dựng hệ thống phân tích trận đấu bóng đá tự động từ video broadcast đơn camera, giải quyết bốn thách thức chính: chuyển động camera mạnh, ánh xạ tọa độ pixel sang sân thực, mất dấu cầu thủ và phân biệt hai đội.

Pipeline tích hợp *YOLOv11s* để phát hiện đối tượng, *ByteTrack* để theo dõi, *OSNet* để tái định danh 5 tầng, quang luồng *Lucas-Kanade* để bù trừ chuyển động camera, *Homography* động để ánh xạ vị trí sang bản đồ 2D, và *K-Means* để phân đội theo màu áo. Hệ thống tính toán tốc độ, quãng đường, tỷ lệ kiểm soát bóng và hiển thị bản đồ chiến thuật dạng *Picture-in-Picture*.

Mô hình đạt **mAP@0.5 = 0.850**, trong đó ba lớp player/referee/goalkeeper vượt AP 0.95; lớp ball còn hạn chế (AP = 0.489) do kích thước nhỏ, được bù bằng nội suy quỹ đạo. Hạn chế chính là hoán đổi định danh còn nhiều và sai số tốc độ khi camera giật đột ngột.

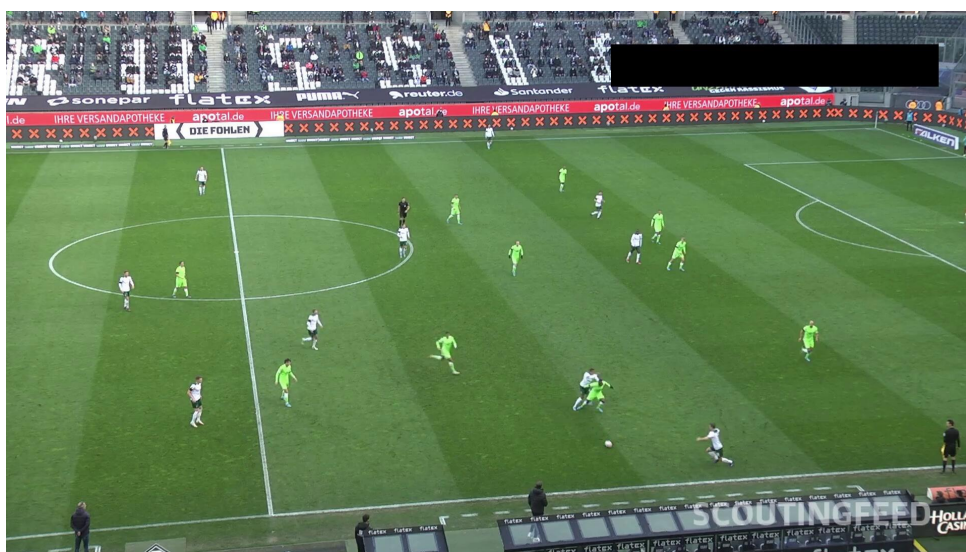
1 Chương 1: Giới Thiệu

1.1 Bối cảnh và vấn đề đặt ra

Bóng đá là môn thể thao vua với lượng dữ liệu hình ảnh khổng lồ được sản sinh sau mỗi trận đấu. Trong những năm gần đây, phân tích dữ liệu thể thao (*sports analytics*) đã trở thành một công cụ không thể thiếu đối với các huấn luyện viên, nhà phân tích chiến thuật và bộ phận tuyển

trạch. Việc nắm bắt chính xác vị trí cầu thủ, quãng đường di chuyển, tốc độ chạy, tỷ lệ kiểm soát bóng và sơ đồ đội hình giúp đội bóng tối ưu hóa lối chơi, đánh giá thể lực và đưa ra các quyết định chiến thuật hợp lý.

Tuy nhiên, ở phần lớn các giải đấu nghiệp dư và bán chuyên, công tác phân tích này hiện nay vẫn được thực hiện một cách thủ công: nhà phân tích phải xem đi xem lại băng hình, tự tay ghi chép vị trí và ước lượng số liệu bằng mắt thường. Phương pháp này tiêu tốn rất nhiều thời gian, dễ sai lệch và khó tái lập. Trong khi đó, các hệ thống thương mại chuyên dụng (sử dụng nhiều camera gắn cố định quanh sân kết hợp cảm biến) lại có chi phí đầu tư cực kỳ lớn, vượt quá khả năng của đa số câu lạc bộ.



Hình 1.1: Khung hình broadcast điển hình của một trận bóng đá

Xuất phát từ thực tế đó, đề án hướng tới việc xây dựng một hệ thống phân tích bóng đá tự động chỉ dựa trên **video quay từ một camera broadcast thông thường**, không yêu cầu thiết bị cảm biến đắt tiền hay nhiều camera đồng bộ. Bài toán này đặt ra bốn thách thức kỹ thuật cốt lõi trong lĩnh vực Thị giác Máy tính:

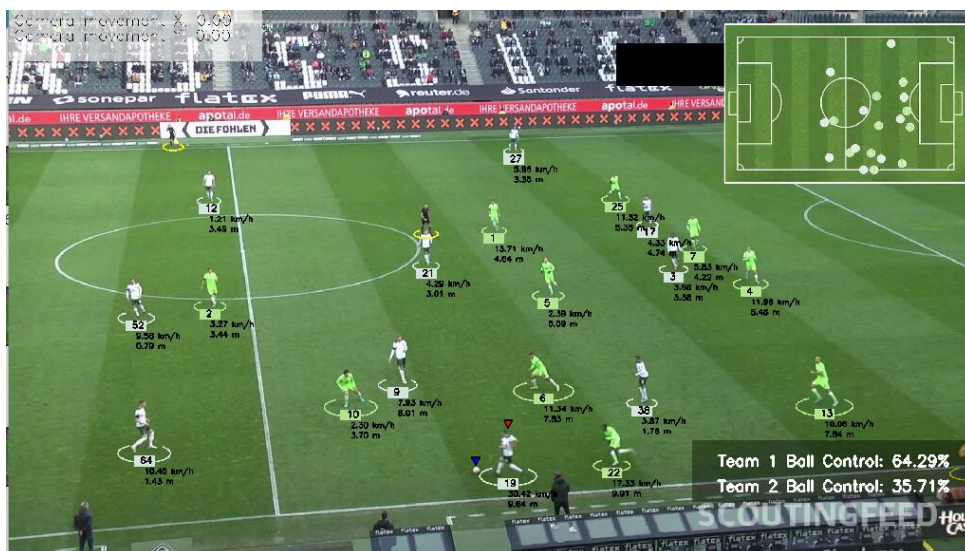
- **Chuyển động camera mạnh và liên tục:** Khác với camera giám sát cố định, camera broadcast liên tục lia (*pan*) và phóng to/thu nhỏ (*zoom*) để bám theo diễn biến trận đấu. Điều này khiến cùng một điểm cố định trên sân lại có tọa độ pixel khác nhau giữa các khung hình, gây sai lệch nghiêm trọng khi tính toán vị trí và tốc độ thực tế nếu không được bù trừ.
- **Ánh xạ tọa độ pixel sang tọa độ thực:** Hình ảnh thu được từ camera là góc nhìn phối cảnh (*perspective view*), trong khi mọi phép đo lường chiến thuật (khoảng cách, tốc độ, đội hình) đều cần được thực hiện trên hệ tọa độ mặt sân thực (đơn vị mét). Việc chuyển đổi này đòi hỏi phép biến đổi phối cảnh (*Homography*), và do camera luôn di chuyển nên ma trận biến đổi phải được ước lượng **động theo từng khung hình**.

- **Mất dấu và hoán đổi định danh (ID Switch):** Trên sân luôn có nhiều cầu thủ di chuyển với tốc độ cao, thường xuyên chạy cắt mặt, va chạm và che khuất lẫn nhau. Các thuật toán tracking truyền thống dễ mất dấu và cấp phát một định danh (ID) mới khi cầu thủ tái xuất hiện, làm gây quỹ đạo và sai lệch số liệu thống kê cá nhân.
- **Phân biệt hai đội bóng trên cùng một sân:** Để tính kiểm soát bóng và dựng sơ đồ chiến thuật, hệ thống phải tự động phân loại cầu thủ thuộc đội nào. Đặc trưng phân biệt chính là màu áo đấu, nhưng điều kiện ánh sáng sân vận động, bóng đổ và độ phân giải thấp khiến việc phân cụm màu trở nên không hề đơn giản.

1.2 Mục tiêu đề tài

Đề án hướng tới việc nghiên cứu và triển khai một pipeline hoàn chỉnh nhằm giải quyết các thách thức nêu trên, với các mục tiêu cụ thể sau:

- **Phát hiện và theo dõi đa đối tượng:** Phát hiện và duy trì định danh ổn định cho ba nhóm đối tượng trong trận đấu — cầu thủ (*player*), trọng tài (*referee*) và bóng (*ball*) — xuyên suốt các khung hình video.
- **Tái định danh cầu thủ (Re-ID):** Giảm thiểu hiện tượng cấp phát ID mới, khôi phục đúng danh tính cầu thủ sau khi bị mất dấu do che khuất hoặc ra khỏi khung hình, dựa trên đặc trưng ngoại hình kết hợp ràng buộc vật lý và vị trí trên sân.
- **Bù trừ chuyển động camera:** Ước lượng độ dịch chuyển của camera giữa các khung hình bằng kỹ thuật quang luồng (*optical flow*), từ đó hiệu chỉnh lại tọa độ đối tượng để phản ánh đúng chuyển động thực của cầu thủ.
- **Biến đổi phối cảnh sang bản đồ 2D:** Ánh xạ vị trí cầu thủ và bóng từ góc nhìn camera sang hệ tọa độ sân thực ($120m \times 70m$) bằng phép biến đổi Homography động, ước lượng theo từng khung hình dựa trên các điểm mốc (*keypoints*) của sân.
- **Ước lượng tốc độ và quãng đường:** Tính toán tốc độ tức thời (km/h) và tổng quãng đường di chuyển (mét) của từng cầu thủ dựa trên dịch chuyển vị trí thực theo thời gian.
- **Phân đội và tính kiểm soát bóng:** Tự động phân cụm màu áo bằng K-Means để gán cầu thủ vào hai đội, sau đó xác định cầu thủ đang giữ bóng và tính tỷ lệ kiểm soát bóng (*ball possession*) của mỗi đội.
- **Trực quan hóa chiến thuật:** Sinh bản đồ chiến thuật 2D dạng Picture-in-Picture (PiP) hiển thị vị trí cầu thủ/bóng cùng vết di chuyển (*trail*), kèm bản đồ nhiệt (*heatmap*) phản ánh khu vực hoạt động.
- **Phân tích và báo cáo tự động:** Tổng hợp số liệu thống kê và sinh báo cáo chiến thuật tự động (PDF) dựa trên dữ liệu đã trích xuất.



Hình 1.2: Tổng quan đầu ra của hệ thống: theo dõi đối tượng, tốc độ/quãng đường, kiểm soát bóng và bản đồ chiến thuật 2D

1.3 Phạm vi

Để đảm bảo tính khả thi và tập trung sâu vào cốt lõi kỹ thuật, phạm vi của hệ thống được giới hạn trong các điều kiện sau:

- **Nguồn đầu vào:** Video bóng đá quay từ **một camera broadcast góc cao** (*broadcast/tactical view*), xử lý ở chế độ offline từ file MP4. Chưa xử lý nguồn đa camera hay luồng trực tiếp thời gian thực.
- **Đối tượng theo dõi:** Chỉ theo dõi cầu thủ, trọng tài và bóng. Thủ môn (*goalkeeper*) được gộp chung vào nhóm cầu thủ trong quá trình xử lý.
- **Kích thước sân:** Áp dụng cho sân bóng tiêu chuẩn với kích thước mô hình hóa 120m (chiều dài) $\times$ 70m (chiều rộng), cùng hệ thống điểm mốc keypoint chuẩn của sân bóng.
- **Số đội:** Hệ thống phân biệt hai đội dựa trên màu áo. Chưa xử lý các trường hợp đặc biệt như hai đội mặc áo gần giống màu, hay nhận diện riêng thủ môn theo màu áo khác biệt.
- **Nền tảng phần cứng:** Hệ thống được thiết kế để vận hành trên môi trường CPU phổ thông (không bắt buộc GPU chuyên dụng), đánh đổi bằng việc xử lý theo lô (*batch*) ở chế độ offline thay vì thời gian thực.
- **Mục tiêu phân tích:** Tập trung vào các chỉ số chiến thuật cơ bản (vị trí, tốc độ, quãng đường, kiểm soát bóng, heatmap). Các bài toán nâng cao như nhận diện hành động (chuyền, sút, tắc bóng) hay phát hiện việt vị thuộc về hướng phát triển tương lai.

2 Chương 2: Cơ Sở Lý Thuyết

2.1 Phát hiện đối tượng: YOLOv11s

2.1.1 Kiến trúc tổng quát của YOLO



Hình 2.1: Person detection bằng YOLO

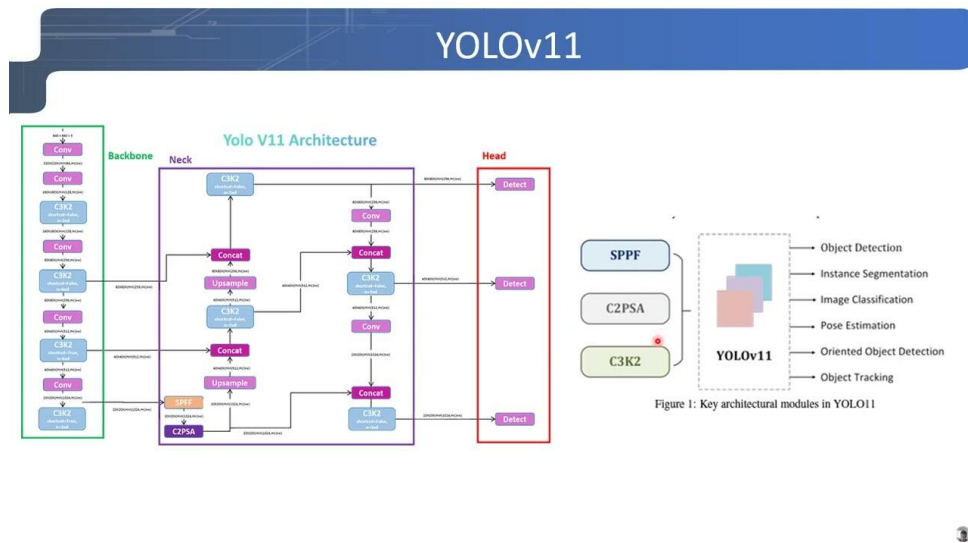
YOLO (*You Only Look Once*) là lớp mô hình phát hiện vật thể một giai đoạn (*one-stage detector*). Mô hình này xử lý toàn bộ ảnh đầu vào chỉ trong một lần lan truyền tiến (*forward pass*) duy nhất qua mạng thần kinh nhân tạo để đồng thời dự đoán tọa độ hộp bao (*bounding box*) và xác suất phân loại thực thể. Kiến trúc tổng quát của YOLO bao gồm ba thành phần cốt lõi:

- **Backbone (Mạng xương sống):** Chịu trách nhiệm trích xuất các đặc trưng phân tầng từ ảnh thô đầu vào (ví dụ: các kiến trúc dựa trên CSPDarknet hoặc C2f/C3k2).
- **Neck (Mạng cổ):** Thực hiện nhiệm vụ tổng hợp và kết hợp các đặc trưng đa tỷ lệ từ các tầng mạng khác nhau (thường sử dụng cấu trúc FPN hoặc PAN), giúp mô hình có khả năng nhận diện tốt vật thể ở các kích thước khác nhau.
- **Head (Đầu dự đoán):** Chịu trách nhiệm tính toán và đưa ra tọa độ hộp bao cùng xác suất của từng lớp đối tượng. Từ phiên bản YOLOv8 trở đi, phần Head được thiết kế theo cơ chế không phụ thuộc vào hộp neo (*anchor-free*).

2.1.2 Mô hình YOLOv11s

Mô hình **YOLOv11s** là phiên bản kích cỡ nhỏ (*small*) thuộc thể hệ YOLOv11 do Ultralytics phát triển, sở hữu dung lượng tham số tối ưu (khoảng 2.6 triệu tham số). Phiên bản này mang lại những cải tiến mang tính bước ngoặt so với thể hệ tiền nhiệm YOLOv8:

- **Module C3k2:** Cấu trúc *Cross Stage Partial* tích hợp nhân convolve kích thước kép được sử dụng để thay thế hoàn toàn cho module C2f trong cấu trúc Backbone, giúp tăng cường khả năng trích xuất các đặc trưng cục bộ phức tạp.
- **Cơ chế chú ý CBAM (Convolutional Block Attention Module):** Tích hợp mạng chú ý kênh và không gian, giúp mô hình tự động tập trung tài nguyên tính toán vào các vùng pixel chứa thông tin quan trọng (thân người) thay vì nhiễu nền studio.
- **Tối ưu hóa phần cứng:** Đạt trạng thái cân bằng xuất sắc giữa độ chính xác và tốc độ xử lý đồ họa, tạo điều kiện lý tưởng cho việc vận hành trực tiếp trên kiến trúc CPU phổ thông.



Hình 2.2: YOLOv11 Architecture

2.2 Theo dõi đa đối tượng: ByteTrack

2.2.1 Bài toán MOT và Bộ lọc Kalman

Bài toán theo dõi đa đối tượng (*Multi-Object Tracking – MOT*) được định nghĩa như sau: Cho trước một chuỗi các khung hình video $\{x_1, x_2, \dots, x_T\}$ và tập hợp các hộp bao phát hiện được D_t tại mỗi thời điểm t . Mục tiêu là tìm ra tập hợp các quỹ đạo chuyển động $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_K\}$ sao cho mỗi thực thể duy trì duy nhất một định danh (ID) nhất quán qua dòng thời gian.

Trong không gian hai chiều của camera, trạng thái của một đối tượng được biểu diễn dưới dạng vector trạng thái liên tục:

$$s = (c_x, c_y, w, h, v_{cx}, v_{cy}, v_w, v_h)^T$$

Bộ lọc Kalman thực hiện hai bước tại mỗi frame:

Bước Predict:

$$\hat{s}_{t|t-1} = F \cdot s_{t-1|t-1}$$

$$P_{t|t-1} = F \cdot P_{t-1|t-1} \cdot F^T + Q$$

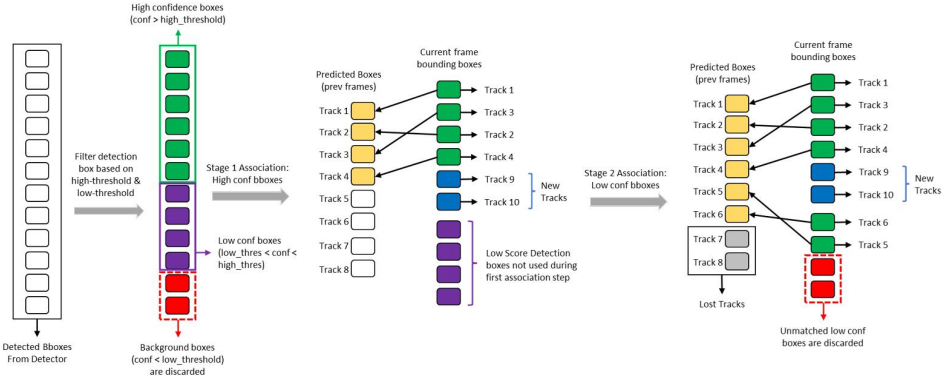
Bước Update:

$$K = P_{t|t-1} \cdot H^T \cdot (H \cdot P_{t|t-1} \cdot H^T + R)^{-1}$$

$$s_{t|t} = \hat{s}_{t|t-1} + K \cdot (z_t - H \cdot \hat{s}_{t|t-1})$$

$$P_{t|t} = (I - K \cdot H) \cdot P_{t|t-1}$$

Trong đó: F = ma trận chuyển trạng thái, Q = ma trận nhiễu quá trình, H = ma trận quan sát, R = ma trận nhiễu quan sát, K = Kalman Gain.



Hình 2.3: ByteTrack Algorithm

2.2.2 Thuật toán Hungarian và Cơ chế Khớp kép (Double Matching)

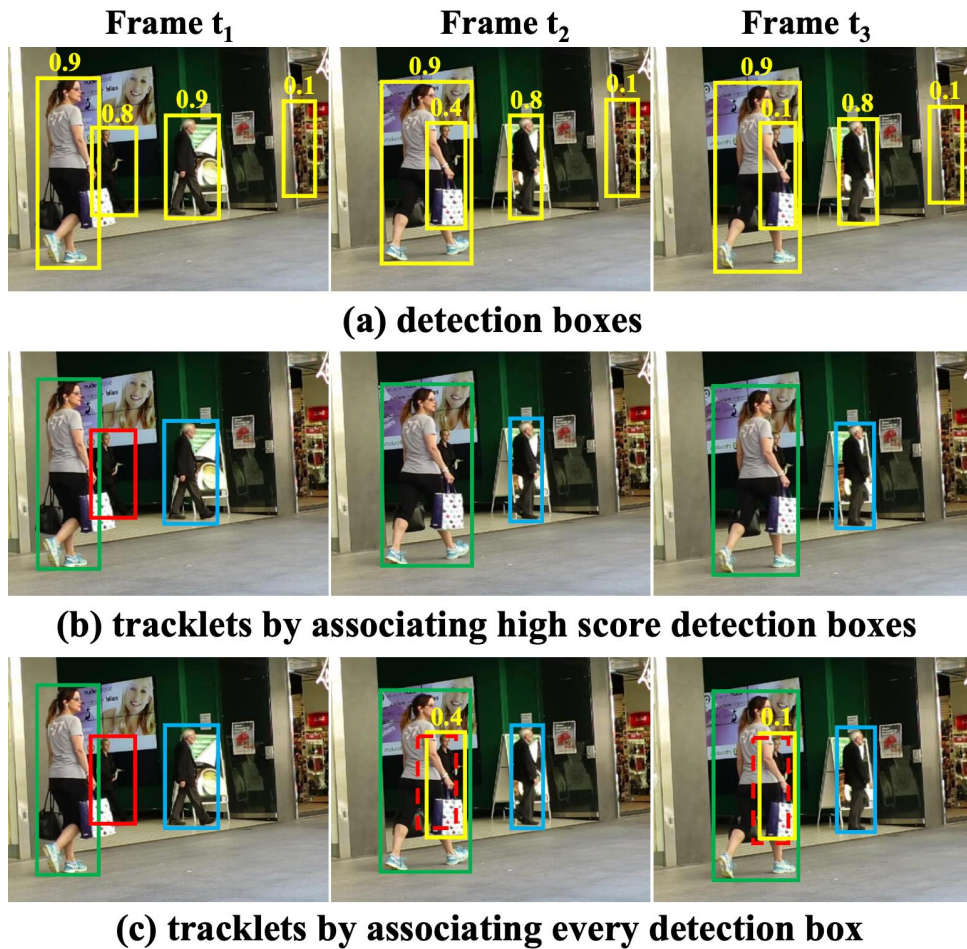
Thuật toán Hungarian được áp dụng nhằm giải quyết bài toán phân công tối ưu giữa tập hợp các thực thể đang được theo dõi (*tracks*) và tập hợp các hộp bao mới phát hiện từ YOLO. Hàm chi phí liên kết sử dụng giá trị khoảng cách hình học:

$$\text{Cost} = 1 - \text{IoU}(\text{box}_{\text{predict}}, \text{box}_{\text{detect}})$$

Điểm đột phá của ByteTrack nằm ở chiến lược **Khớp kép (Double Matching)**, giúp tận dụng toàn bộ các hộp phát hiện thay vì loại bỏ các hộp bao có điểm tin cậy thấp:

- **Giai đoạn 1:** Kết hợp high-confidence box với active tracklets bằng IoU matching (Hungarian algorithm) + Kalman Filter dự đoán.
- **Giai đoạn 2:** Kết hợp low-confidence box còn lại với lost tracklets để hồi phục track bị mất.

Trong đồ án, ByteTrack được cấu hình với `lost_track_buffer = 130 frames` (tương đương ≈ 10 giây video gốc) để chịu được khoảng dừng dài hơn trước khi ReID xử lý.



Hình 2.4: ByteTrack sử dụng cả low-confidence và high-confidence bounding boxes

2.3 Tái định danh người: OSNet

2.3.1 Bài toán Person Re-Identification (ReID)

Tái định danh người (*Person ReID*) về bản chất là một bài toán học tập khoảng cách (*metric learning*). Mục tiêu là huấn luyện một mạng nơ-ron sâu trích xuất vector đặc trưng diện mạo sao cho khoảng cách giữa ảnh của cùng một cá nhân là tối thiểu, và khoảng cách giữa các cá nhân khác nhau là tối đa.

Trong không gian quản lý của studio, module ReID phải đối mặt trực tiếp với 3 bài toán đặc thù:

- **Sự biến đổi ngoại hình cực đại:** Khách hàng thực hiện cởi/thay áo khoác hoặc đổi trang phục khiến cấu trúc màu sắc chủ đạo trên cơ thể thay đổi hoàn toàn.
- **Mất dấu thời gian dài (*Long-term disappearance*):** Khách hàng rời khỏi khu vực giám sát để đi vệ sinh hoặc phòng thay đồ độc lập trong thời gian từ 5 – 10 phút, vượt quá ngưỡng lưu giữ hình học của mạng tracking.

- **Hiện tượng che khuất diện rộng:** Mật độ người tập trung tại các khu vực tiểu cảnh hoặc bàn trang điểm dễ gây ra hiện tượng giao thoa góc nhìn camera.

2.3.2 OSNet – Omni-Scale Feature Learning

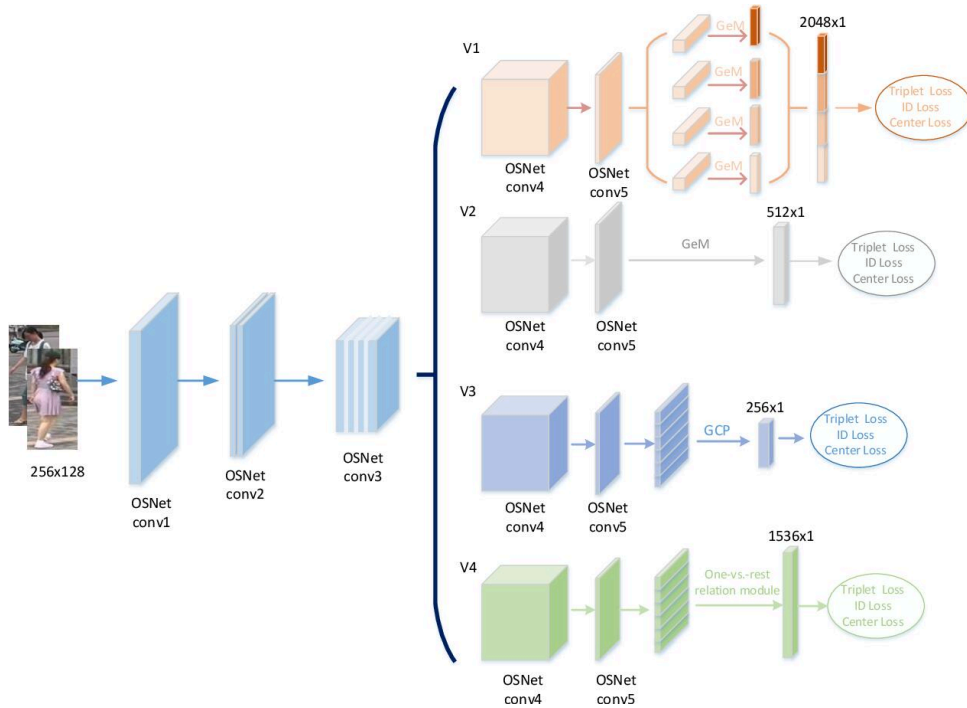
Mạng **OSNet** (*Omni-Scale Network*) được thiết kế chuyên biệt để trích xuất các đặc trưng đa tỷ lệ đồng thời (*omni-scale features*) thông qua cấu trúc khối thặng dư cải tiến mang tên **OS Block**:

$$f_{\text{out}} = \sum_i AG_i(x) \odot T_i(x)$$

Trong đó:

- $T_i(x)$: đầu ra nhánh trích xuất đặc trưng thứ i sử dụng các kích thước nhân convolution khác nhau ($1 \times 1, 3 \times 3, 5 \times 5, \dots$).
- $AG_i(x)$: trọng số phân bố được tính toán từ tổng hợp nhất thống nhất, tự động tối ưu hóa qua quá trình học tập.
- $\odot$: phép nhân từng phần tử (element-wise multiplication).

Nhằm triển khai mượt mà trên môi trường CPU, hệ thống ứng dụng cấu trúc Tích chập phân tách chiều sâu (Depthwise Separable Convolution) xuyên suốt các nhánh đa tỷ lệ.



Hình 2.5: Branch-Cooperative OSNet for Person Re-Identification

2.3.3 Hàm mất mát huấn luyện

$$L_{\text{total}} = L_{\text{ID}} + L_{\text{triplet}}$$

- L_{ID} : Softmax Cross-Entropy Loss (phân loại từng người như một lớp).
- L_{triplet} : Triplet Loss buộc không gian embedding co giãn đúng chiều:

$$L_{\text{triplet}} = \max(d(a, p) - d(a, n) + \text{margin}, 0)$$

Trong đó a = anchor, p = positive sample (cùng người), n = negative sample (người khác), $\text{margin} = 0.3$, d = khoảng cách Euclidean trong không gian đặc trưng.

2.3.4 Độ tương đồng Cosine và ngưỡng quyết định

$$S_{\cos}(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

Lưu ý kỹ thuật: Thay vì so sánh trực tiếp với một khung hình đơn lẻ dễ bị ảnh hưởng bởi nhiễu ánh sáng, hệ thống sử dụng **Vector đặc trưng trung bình (Mean Feature Vector)** được tích lũy liên tục qua quy tắc 10 khung hình (*10-frame rule*) gần nhất nhằm đảm bảo tính ổn định tối đa cho quá trình nhận diện thực tế.

2.4 Biến đổi phối cảnh: Homography

Phép biến đổi hình học phẳng (*Homography*) đóng vai trò thiết lập ánh xạ tọa độ từ hệ quy chiếu phối cảnh nghiêng mang tính biến dạng của camera giám sát về mặt phẳng có góc nhìn từ trên xuống (*top-down view*) của sơ đồ studio.

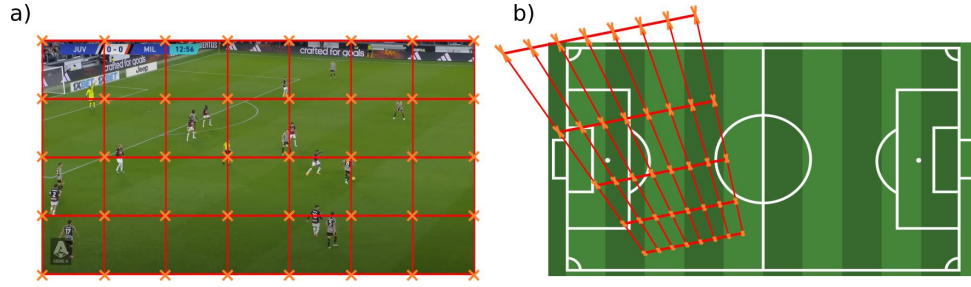
Phương trình biến đổi tọa độ đồng nhất:

$$s \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Tọa độ thực tế (x', y') trên bản đồ phẳng top-down được trích xuất qua các hệ thức phi tuyến:

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}, \quad y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}$$

Đồ án chia studio thành hai ROI (upper và lower), mỗi ROI có ma trận H riêng, xử lý tốt hơn hiệu ứng phối cảnh khi studio có chiều sâu lớn.



Hình 2.6: Homography Transformations

2.5 Ước lượng chuyển động camera: Optical Flow

Trong video bóng đá broadcast, camera liên tục lia và phóng to/thu nhỏ để bám theo bóng. Hệ quả là một điểm cố định trên sân (ví dụ góc khung thành) lại có tọa độ pixel thay đổi giữa các khung hình. Nếu không bù trừ, độ dịch chuyển pixel do camera gây ra sẽ bị nhầm thành chuyển động thực của cầu thủ, làm sai lệch tốc độ và quãng đường. Để giải quyết, hệ thống sử dụng kỹ thuật **quang luồng (Optical Flow)** theo thuật toán Lucas–Kanade.

2.5.1 Giả định độ sáng bất biến

Optical Flow dựa trên giả định rằng cường độ sáng của một điểm ảnh không thay đổi giữa hai khung hình liên tiếp khi điểm đó dịch chuyển một lượng nhỏ (dx, dy) trong khoảng thời gian dt :

$$I(x, y, t) = I(x + dx, y + dy, t + dt)$$

Khai triển Taylor bậc nhất về phải và bỏ qua các số hạng bậc cao, ta thu được **phương trình ràng buộc quang luồng**:

$$I_x \cdot u + I_y \cdot v + I_t = 0$$

Trong đó: I_x, I_y là đạo hàm cường độ theo phương ngang và dọc, I_t là đạo hàm theo thời gian, còn $(u, v) = \left(\frac{dx}{dt}, \frac{dy}{dt}\right)$ là vector vận tốc quang luồng cần tìm.

2.5.2 Giải bằng phương pháp Lucas–Kanade

Một phương trình với hai ẩn (u, v) là bài toán thiếu xác định (*aperture problem*). Lucas–Kanade khắc phục bằng giả định rằng mọi điểm ảnh trong một cửa sổ nhỏ W ($n \times n$ pixel) có cùng vector chuyển động. Khi đó ta có hệ phương trình thừa xác định, giải bằng bình phương tối thiểu:

$$\begin{bmatrix} u \\ v \end{bmatrix} = (A^T A)^{-1} A^T b$$

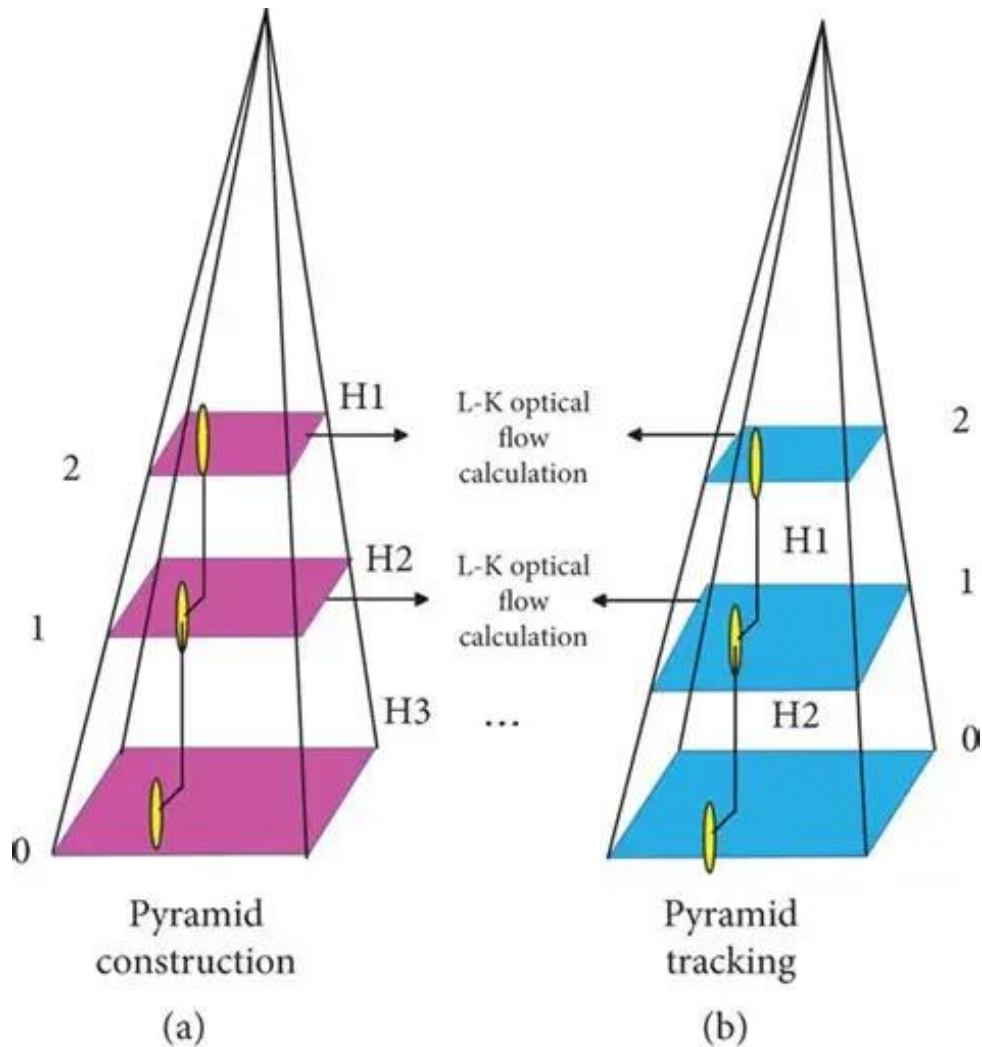
với

$$A = \begin{bmatrix} I_{x_1} & I_{y_1} \\ \vdots & \vdots \\ I_{x_k} & I_{y_k} \end{bmatrix}, \quad b = - \begin{bmatrix} I_{t_1} \\ \vdots \\ I_{t_k} \end{bmatrix}$$

trong đó $k = n^2$ là số điểm ảnh trong cửa sổ W .

2.5.3 Phiên bản kim tự tháp (Pyramidal LK)

Để xử lý chuyển động lớn của camera, thuật toán áp dụng cấu trúc kim tự tháp ảnh đa độ phân giải (*image pyramid*). Chuyển động được ước lượng từ tầng độ phân giải thấp nhất (chuyển động nhỏ hơn) rồi tinh chỉnh dần lên các tầng độ phân giải cao hơn. Trong đồ án, tham số được cấu hình `winSize = (15, 15)` và `maxLevel = 2`.



Hình 2.7: kim tự tháp (Pyramidal LK)

2.5.4 Ước lượng và bù trừ chuyển động

Hệ thống chỉ trích đặc trưng góc (*good features to track*) ở hai dải biên trái/phải của khung hình — nơi ít cầu thủ xuất hiện nên đại diện tốt cho nền sân cỏ. Độ dịch chuyển camera của khung hình được lấy theo điểm đặc trưng có độ dời lớn nhất:

$$d_{\max} = \max_i \sqrt{(x_i^{\text{new}} - x_i^{\text{old}})^2 + (y_i^{\text{new}} - y_i^{\text{old}})^2}$$

Chuyển động camera chỉ được ghi nhận khi $d_{\max}$ vượt ngưỡng tối thiểu (`min_distance = 5 pixel`) nhằm loại bỏ nhiễu. Sau đó, vị trí mỗi đối tượng được hiệu chỉnh bằng cách trừ đi lượng dịch chuyển camera:

$$p_{\text{adjusted}} = (x - \Delta x_{\text{cam}}, y - \Delta y_{\text{cam}})$$

Tọa độ đã hiệu chỉnh p_{adjusted} mới được dùng cho bước biến đổi phối cảnh ở giai đoạn sau.

2.6 Phân cụm màu áo: Thuật toán K-Means

Để tính tỷ lệ kiểm soát bóng và dựng sơ đồ chiến thuật, hệ thống phải phân loại mỗi cầu thủ thuộc đội nào. Đặc trưng phân biệt chính là màu áo đấu, do đó đồ án sử dụng thuật toán phân cụm **K-Means** để gom màu áo thành hai cụm tương ứng hai đội.

2.6.1 Bài toán phân cụm

Cho tập hợp N điểm dữ liệu $\{x_1, x_2, \dots, x_N\}$ trong không gian màu (mỗi điểm là một vector RGB 3 chiều), K-Means chia chúng thành k cụm sao cho tổng bình phương khoảng cách từ mỗi điểm tới tâm cụm của nó là nhỏ nhất:

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

Trong đó C_i là cụm thứ i , μ_i là tâm (centroid) của cụm C_i , và $\|\cdot\|$ là khoảng cách Euclidean. Với bài toán phân đội, $k = 2$.

2.6.2 Quy trình lặp

Thuật toán tối ưu hàm mục tiêu J qua hai bước lặp xen kẽ cho tới khi hội tụ:

- **Bước gán (Assignment):** Gán mỗi điểm vào cụm có tâm gần nhất.

$$C_i = \{x : \|x - \mu_i\|^2 \leq \|x - \mu_j\|^2, \forall j\}$$

- **Bước cập nhật (Update):** Tính lại tâm mỗi cụm bằng trung bình các điểm thuộc cụm.

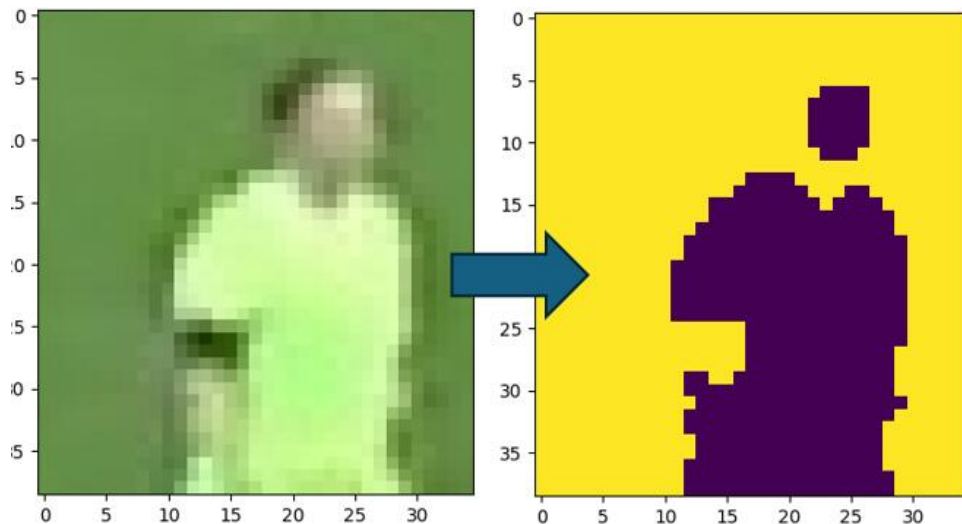
$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

Để tăng tốc độ hội tụ và tránh kẹt ở cực tiểu cục bộ, đồ án sử dụng phương pháp khởi tạo k-means++.

2.6.3 Trích xuất màu áo cầu thủ

Quy trình gán đội cho mỗi cầu thủ gồm các bước:

1. Cắt vùng ảnh bên trong bounding box của cầu thủ, sau đó chỉ lấy **nửa trên** của bounding box — vùng chứa áo đầu, loại bỏ phần quần và chân.
2. Áp dụng K-Means với $k = 2$ trên các pixel của vùng áo để tách **pixel áo** khỏi **pixel nền sân cỏ**. Cụm chiếm các góc ảnh được xác định là nền (vì viền bounding box thường là cỏ), cụm còn lại là màu áo.
3. Thu thập màu áo đại diện của tất cả cầu thủ trong khung hình đầu, rồi chạy K-Means lần hai (cũng với $k = 2$) trên tập màu áo này để xác định **hai màu đại diện cho hai đội**.
4. Mỗi cầu thủ mới được gán đội bằng cách dự đoán cụm mà màu áo của họ thuộc về.



Hình 2.8: Dùng kmean để trích xuất đặc trưng

2.7 Ước lượng tốc độ và quãng đường

Sau khi vị trí cầu thủ đã được ánh xạ sang tọa độ sân thực (đơn vị mét) qua phép biến đổi Homography, hệ thống ước lượng tốc độ và quãng đường di chuyển dựa trên động học cơ bản.

2.7.1 Quãng đường di chuyển

Trong một cửa sổ thời gian gồm w khung hình (đồ án dùng `frame_window = 5`), quãng đường mà cầu thủ di chuyển được tính bằng khoảng cách Euclidean giữa vị trí thực ở khung hình đầu và khung hình cuối của cửa sổ:

$$d = \sqrt{(x_{end} - x_{start})^2 + (y_{end} - y_{start})^2}$$

Tổng quãng đường của một cầu thủ là tổng tích lũy quãng đường qua tất cả các cửa sổ:

$$D_{total} = \sum_j d_j$$

2.7.2 Tốc độ tức thời

Thời gian trôi qua trong cửa sổ được suy ra từ số khung hình và tốc độ khung hình (*frame rate*):

$$\Delta t = \frac{w}{\text{fps}}$$

Tốc độ trung bình trong cửa sổ (mét trên giây) và chuyển đổi sang km/h:

$$v_{m/s} = \frac{d}{\Delta t}, \quad v_{km/h} = v_{m/s} \times 3.6$$

2.8 Gán bóng và tính kiểm soát bóng

Để xác định đội nào đang kiểm soát bóng, hệ thống gán quyền giữ bóng cho cầu thủ gần bóng nhất tại mỗi khung hình.

2.8.1 Gán bóng cho cầu thủ

Với mỗi cầu thủ, hệ thống tính khoảng cách từ tâm bóng tới hai điểm chân (góc dưới trái và góc dưới phải của bounding box) và lấy giá trị nhỏ hơn:

$$d_{player} = \min\left(\text{dist}(p_{ball}, p_{left}), \text{dist}(p_{ball}, p_{right})\right)$$

Cầu thủ có d_{player} nhỏ nhất và nằm dưới ngưỡng cho phép (`max_player_ball_distance = 50 pixel`) được xác định là người đang giữ bóng. Nếu không cầu thủ nào thỏa ngưỡng, khung hình đó không gán quyền giữ bóng.

2.8.2 Tỷ lệ kiểm soát bóng

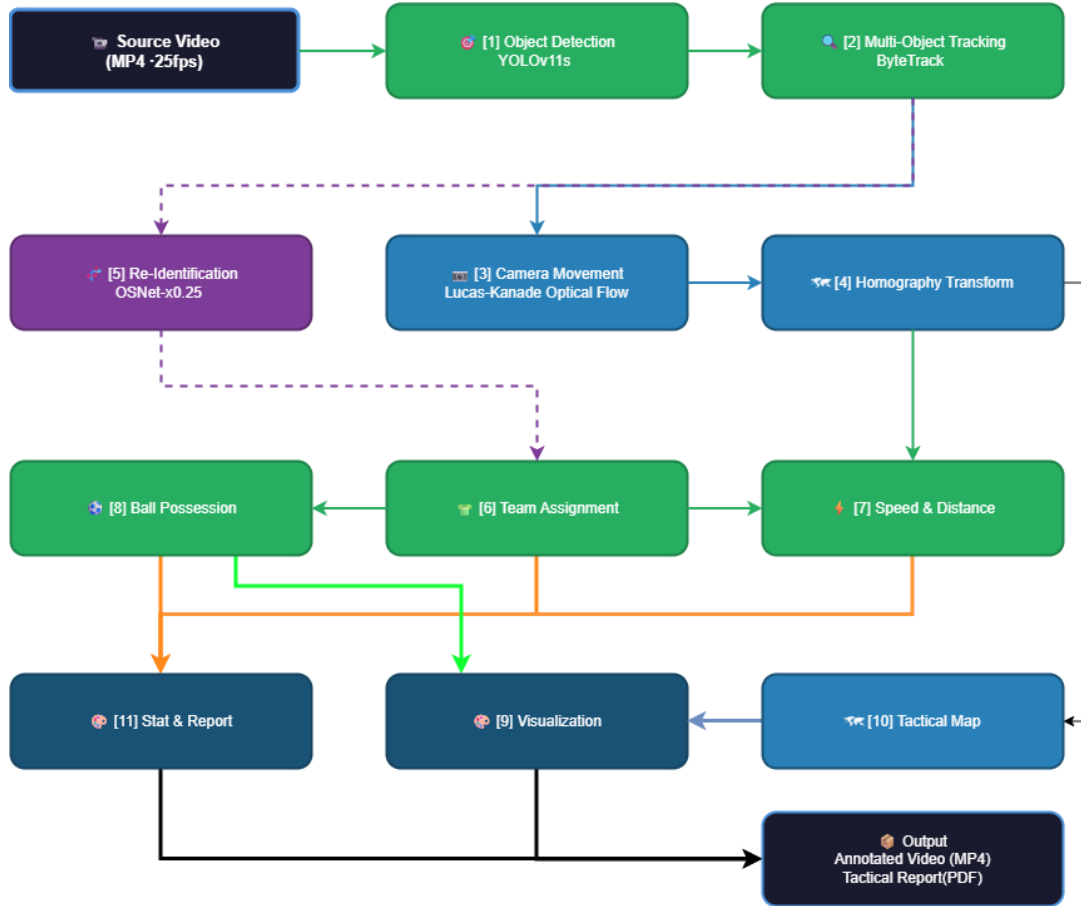
Đội của cầu thủ giữ bóng được ghi nhận tại mỗi khung hình. Tỷ lệ kiểm soát bóng của một đội tính đến khung hình hiện tại là tỷ số giữa số khung hình đội đó giữ bóng và tổng số khung hình

đã ghi nhận:

$$\text{Possession}_{team} = \frac{N_{team}}{N_{team_1} + N_{team_2}} \times 100\%$$

3 Chương 3: Phương Pháp Thực Hiện

3.1 Sơ đồ kiến trúc tổng thể



Hình 3.1: Sơ đồ kiến trúc phân tầng toàn hệ thống

Hệ thống được thiết kế theo kiến trúc pipeline xử lý tuần tự, trong đó đầu ra của mỗi module là đầu vào của module kế tiếp. Toàn bộ luồng xử lý được điều phối tập trung trong `main.py` và bao gồm mười một giai đoạn chính, được thực thi theo thứ tự sau:

1. **Đọc video (Read Video):** Nạp toàn bộ khung hình từ file video đầu vào vào bộ nhớ.
2. **Theo dõi đối tượng (Tracking):** Phát hiện và gán định danh cho cầu thủ, trọng tài, bóng qua YOLOv11 + ByteTrack; đồng thời tính điểm đại diện vị trí (`position`) cho mỗi đối tượng.

3. **Ước lượng chuyển động camera (Camera Movement):** Đo độ dịch chuyển của camera giữa các khung hình bằng optical flow, sau đó hiệu chỉnh vị trí đối tượng để loại bỏ ảnh hưởng của camera.
4. **Biến đổi phối cảnh (View Transform):** Ánh xạ vị trí đã hiệu chỉnh từ tọa độ pixel sang tọa độ sân thực (mét) bằng Homography động theo từng khung hình.
5. **Tái định danh (ReID):** Hợp nhất các định danh bị phân mảnh do mất dấu/che khuất bằng đặc trưng OSNet kết hợp ràng buộc đội, vật lý và vị trí trên sân.
6. **Gán đội (Team Assign):** Phân cụm màu áo bằng K-Means để gán mỗi cầu thủ vào một trong hai đội.
7. **Tốc độ & quãng đường (Speed & Distance):** Tính tốc độ tức thời (km/h) và tổng quãng đường (mét) cho từng cầu thủ.
8. **Kiểm soát bóng (Ball Possession):** Xác định cầu thủ giữ bóng tại mỗi khung hình và tính tỷ lệ kiểm soát bóng của hai đội.
9. **Vẽ chú thích (Draw Annotations):** Vẽ ellipse + ID, tam giác chỉ bóng, ô tỷ lệ kiểm soát bóng, thông tin camera và tốc độ/quãng đường lên khung hình gốc.
10. **Bản đồ chiến thuật (Tactical Map PiP):** Dựng bản đồ 2D top-down với vị trí và vận động của cầu thủ/bóng, overlay vào góc khung hình.
11. **Tổng hợp thống kê & báo cáo (Stats & Report):** Xuất số liệu thống kê, heatmap và báo cáo chiến thuật.

3.2 Mô tả bài toán

3.2.1 Đầu vào (Input)

Đầu vào của hệ thống là một video bóng đá, được biểu diễn dưới dạng chuỗi T khung hình liên tiếp:

$$V = \{F_t\}_{t=1}^T, \quad F_t \in \mathbb{R}^{H \times W \times 3}$$

Trong đó:

- V : chuỗi khung hình của video đầu vào.
- T : tổng số khung hình.
- t : chỉ số khung hình, $t \in \{1, 2, \dots, T\}$.
- F_t : khung hình thứ t , là một ảnh màu.

- H, W : chiều cao và chiều rộng khung hình (pixel).
- 3: số kênh màu (BGR/RGB).

3.2.2 Đầu ra (Output)

Tại mỗi khung hình F_t , hệ thống trả về tập kết quả mô tả trạng thái của toàn bộ đối tượng trên sân:

$$R_t = \{ (id_j, b_t^j, p_t^j, v_t^j, team_j) \}_{j=1}^{n_t}$$

Trong đó:

- R_t : tập kết quả tại khung hình t .
- n_t : số đối tượng hiện diện tại khung hình t .
- id_j : định danh duy nhất của đối tượng thứ j sau khi qua ReID.
- $b_t^j = (x_1, y_1, x_2, y_2) \in \mathbb{R}^4$: bounding box trên khung hình camera.
- $p_t^j = (x, y) \in \mathbb{R}^2$: vị trí được ánh xạ lên bản đồ sân 2D thực (đơn vị mét) qua Homography.
- $v_t^j \in \mathbb{R}^+$: tốc độ tức thời (km/h) của đối tượng (chỉ áp dụng cho cầu thủ).
- $team_j \in \{1, 2\}$: đội mà cầu thủ thuộc về (chỉ áp dụng cho cầu thủ).

Ngoài kết quả theo từng khung hình, hệ thống còn tổng hợp các chỉ số cấp độ trận đấu:

- **Tỷ lệ kiểm soát bóng** của mỗi đội (tích lũy đến từng khung hình).
- **Tổng quãng đường** di chuyển của từng cầu thủ.
- **Bản đồ chiến thuật 2D** và **bản đồ nhiệt** phản ánh phân bố không gian.

3.2.3 Minh họa



Hình 3.2: Minh họa đầu vào (khung hình gốc) và đầu ra (khung hình đã chú thích kèm bản đồ chiến thuật 2D)

3.3 Dataset và huấn luyện mô hình

Hệ thống sử dụng **hai mô hình học sâu độc lập**, mỗi mô hình đảm nhận một nhiệm vụ riêng và được huấn luyện trên hai bộ dữ liệu khác nhau. Việc tách thành hai mô hình giúp mỗi mạng tập trung tối ưu cho một bài toán cụ thể, thay vì gán đồng thời nhiều nhiệm vụ.

3.3.1 Hai mô hình cần huấn luyện

a) Mô hình phát hiện đối tượng (best_yolo11s.pt)

Đây là mô hình YOLOv11s được huấn luyện cho bài toán phát hiện vật thể (*object detection*),

chịu trách nhiệm xác định vị trí các đối tượng trong từng khung hình. Mô hình được huấn luyện trên bốn lớp:

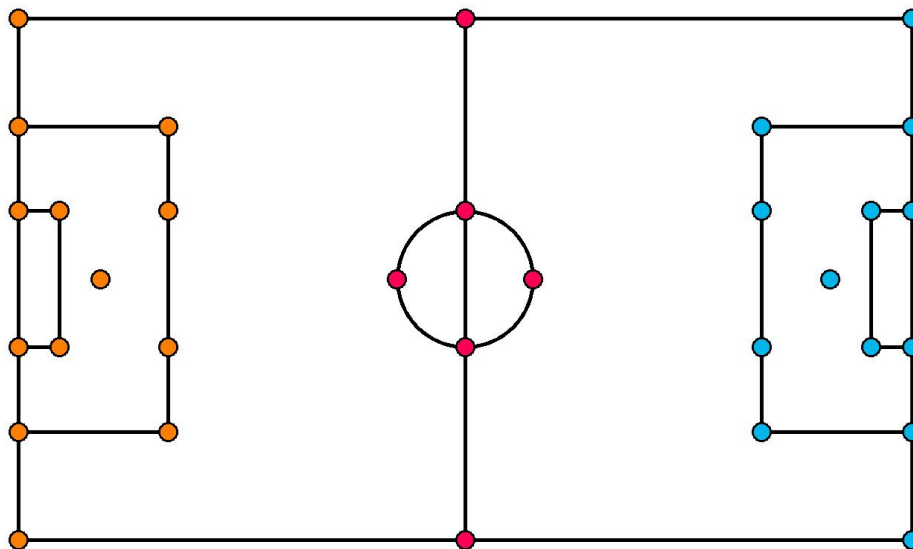
- **player** — cầu thủ ngoài sân.
- **goalkeeper** — thủ môn.
- **referee** — trọng tài.
- **ball** — quả bóng.

Riêng lớp **goalkeeper** tuy được gán nhãn và huấn luyện riêng, nhưng trong quá trình xử lý (`tracker.py`) sẽ được gộp chung vào lớp **player**. Lý do là thủ môn về bản chất vẫn là một cầu thủ trên sân; việc tách lớp khi huấn luyện giúp mô hình học tốt hơn đặc trưng của thủ môn (thường mặc áo màu khác biệt, đứng gần khung thành), nhưng khi phân tích chiến thuật thì gộp lại để thống nhất luồng xử lý cầu thủ.

b) Mô hình phát hiện điểm mốc sân (`best1_pitch.pt`)

Đây là mô hình YOLOv11-Pose (*keypoint detection*), chịu trách nhiệm phát hiện các **điểm mốc đặc trưng của sân bóng** (*pitch keypoints*) — chẳng hạn các góc sân, giao điểm của vạch vôi, các góc vòng cấm, tâm vòng tròn giữa sân. Đầu ra của mô hình là tọa độ pixel của các keypoint cùng độ tin cậy (*confidence*) tương ứng.

Các điểm mốc này được sử dụng để ước lượng ma trận Homography động. Hệ thống ghép cặp tọa độ pixel của các keypoint được phát hiện trên ảnh với tọa độ thực tương ứng trên sân. Các tọa độ chuẩn này được xác định trước thông qua 32 điểm mốc của sân bóng. Từ các cặp điểm tương ứng, hệ thống tính toán phép biến đổi Homography nhằm ánh xạ tọa độ từ hệ tọa độ ảnh sang hệ tọa độ sân 2D. Do camera truyền hình liên tục thay đổi vị trí và góc quan sát, quá trình phát hiện keypoint được thực hiện trên từng khung hình để cập nhật ma trận Homography theo thời gian thực.

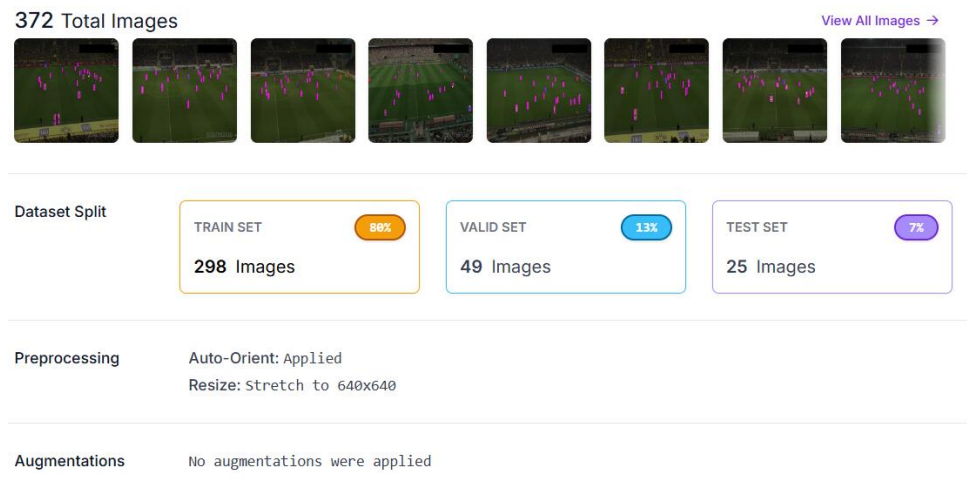


Hình 3.3: Các điểm mốc sân (pitch keypoints) được mô hình YOLO-Pose phát hiện

3.3.2 Thu thập và gán nhãn dữ liệu

Dữ liệu huấn luyện được xây dựng từ các video bóng đá broadcast thực tế. Quy trình chuẩn bị dữ liệu gồm các bước:

1. **Tách khung hình:** Từ các video trận đấu, tiến hành tách khung hình theo tần suất lấy mẫu hợp lý (lấy một ảnh sau mỗi vài chục khung hình) nhằm giảm sự trùng lặp giữa các khung hình liên kề, đồng thời vẫn đảm bảo độ đa dạng về tình huống thi đấu, góc quay và điều kiện ánh sáng.
2. **Gán nhãn trên Roboflow:** Toàn bộ ảnh được gán nhãn thủ công bằng nền tảng **Roboflow** — một công cụ quản lý và gán nhãn dữ liệu trực tuyến chuyên dụng cho Thị giác Máy tính.
 - Với bộ dữ liệu **detection**: vẽ bounding box quanh từng đối tượng và gán vào một trong bốn lớp (player, goalkeeper, referee, ball).
 - Với bộ dữ liệu **keypoint**: đánh dấu vị trí các điểm mốc đặc trưng của sân theo đúng thứ tự quy ước.
3. **Tiền xử lý và tăng cường dữ liệu:** Trước khi huấn luyện, ảnh được chuẩn hóa kích thước (resize về 640×640) và áp dụng các kỹ thuật tăng cường dữ liệu (*data augmentation*) như điều chỉnh độ sáng, thêm nhiễu, lật ảnh... nhằm tăng khả năng tổng quát hóa, giúp mô hình hoạt động ổn định trên nhiều điều kiện sân bãi và camera khác nhau.



Hình 3.4: Gán nhãn dữ liệu bóng đá trên nền tảng Roboflow

3.3.3 Huấn luyện mô hình

Do yêu cầu tính toán lớn khi huấn luyện mô hình học sâu, quá trình huấn luyện được thực hiện trên **Google Colab** với GPU miễn phí (Tesla T4 hoặc tương đương). Thư viện chính được sử dụng là **Ultralytics** — framework cung cấp sẵn các mô hình YOLOv11 cùng pipeline huấn luyện thuận tiện.

Bước 1 — Cài đặt môi trường:

```
!pip install ultralytics roboflow
```

Bước 2 — Tải dữ liệu từ Roboflow:

```
from roboflow import Roboflow
rf = Roboflow(api_key="YOUR_API_KEY")
project = rf.workspace("workspace-name").project("project-name")
dataset = project.version(1).download("yolo11")
```

Bước 3 — Huấn luyện mô hình detection:

```
from ultralytics import YOLO
!yolo task=detect mode=train model=yolo11s.pt \
  data={dataset.location}/data.yaml epochs=100 imgsz=640
```

Bước 4 — Huấn luyện mô hình keypoint sâu:

```
!yolo task=pose mode=train model=yolo11s-pose.pt \
  data={dataset.location}/data.yaml epochs=100 imgsz=640
```

Sau khi huấn luyện, trọng số tốt nhất (best.pt) của mỗi mô hình được lưu lại và đổi tên thành best_yolo11s.pt (detection) và best1_pitch.pt (keypoint), sau đó đặt vào thư mục models/ để pipeline chính nạp khi chạy.

3.4 Module Tracking (`tracker.py`)

Module Tracking là nền tảng của toàn bộ pipeline, chịu trách nhiệm phát hiện và duy trì định danh cho các đối tượng qua từng khung hình. Module kết hợp bộ phát hiện YOLOv11 với thuật toán theo dõi ByteTrack (đóng gói sẵn trong thư viện Supervision).

3.4.1 Phát hiện theo lô (batch)

Để tận dụng khả năng xử lý song song và giảm chi phí khởi tạo cho mỗi lần suy luận, module không phát hiện từng khung hình riêng lẻ mà gom thành các lô (*batch*). Hàm `detect_frames` chia toàn bộ video thành các lô **20 khung hình**, mỗi lô được đưa qua mô hình YOLO một lần với ngưỡng tin cậy `conf = 0.3`. Ngưỡng confidence được đặt tương đối thấp nhằm hạn chế bỏ sót các đối tượng nhỏ hoặc bị che khuất một phần — đặc biệt là quả bóng, vốn rất dễ bị mất dấu.

3.4.2 Gộp lớp goalkeeper vào player

Sau khi YOLO trả về kết quả phát hiện, mô hình phân biệt bốn lớp: player, goalkeeper, referee và ball. Tuy nhiên, ngay trong bước chuyển đổi định dạng, mọi đối tượng được phát hiện thuộc lớp **goalkeeper** đều được gán lại thành lớp **player**.

Việc tách riêng goalkeeper khi huấn luyện giúp mô hình học tốt đặc trưng riêng của thủ môn (áo màu khác biệt, vị trí gần khung thành). Nhưng nếu giữ goalkeeper thành một lớp riêng trong khâu phân tích, nó sẽ gây nhiễu cho các bước phía sau — đặc biệt là module gán đội (Team Assign): thủ môn thường mặc áo màu hoàn toàn khác hai đội, nếu xử lý riêng sẽ làm phức tạp logic phân cụm màu. Do đó, gộp goalkeeper vào player ngay từ đầu giúp thống nhất luồng xử lý: mọi cầu thủ (kể cả thủ môn) đều đi chung một đường ống tracking, gán đội và tính toán chỉ số.

3.4.3 Tích hợp ByteTrack

Sau khi có kết quả phát hiện đã chuẩn hóa, hàm `update_with_detections` của ByteTrack được gọi để gán và duy trì định danh (*track id*) cho các đối tượng qua các khung hình.

Một điểm thiết kế quan trọng là **cách xử lý khác nhau giữa các nhóm đối tượng**:

- **Cầu thủ và trọng tài** được đưa qua ByteTrack để gán track id ổn định. Đây là các đối tượng di chuyển có quỹ đạo, cần duy trì danh tính theo thời gian.
- **Bóng không được đưa qua ByteTrack.** Thay vào đó, bóng được lấy trực tiếp từ kết quả phát hiện thô và luôn gán định danh cố định `id = 1`. Lý do là trên sân chỉ có duy nhất một quả bóng, nên không cần thuật toán theo dõi đa đối tượng; hơn nữa bóng di chuyển rất nhanh và thất thường, việc ép qua mô hình chuyển động Kalman của ByteTrack dễ gây sai lệch quỹ đạo hơn là hữu ích.

3.4.4 Cấu trúc dữ liệu tracks

Toàn bộ thông tin theo dõi được lưu trong một dictionary lồng nhau (tracks), được dùng xuyên suốt pipeline và bồi đắp dần qua từng module. Cấu trúc tổng quát:

```
tracks[object][frame_num][track_id] = {
    "bbox": [x1, y1, x2, y2],
    "position": (x, y),
    "position_adjusted": (x, y),
    "position_transformed": (x_m, y_m),
    "team": int,
    "team_color": (B, G, R),
    "speed": float,
    "distance": float,
    "has_ball": bool
}
```

Trong đó, ba khóa cấp ngoài cùng object là "players", "referees" và "ball". Ý nghĩa các trường:

- **bbox** — bounding box trên khung hình camera.
- **position** — điểm đại diện vị trí (điểm chân với người, tâm với bóng), do bước Tracking sinh ra.
- **position_adjusted** — vị trí sau khi bù trừ chuyển động camera (do module Camera Movement bổ sung).
- **position_transformed** — vị trí thực trên sân 2D, đơn vị mét (do module View Transform bổ sung).
- **team** — đội của cầu thủ (do Team Assign bổ sung; chỉ áp dụng cho cầu thủ).
- **team_color** — màu đại diện của đội, dùng khi vẽ chú thích và bản đồ chiến thuật.
- **speed, distance** — tốc độ (km/h) và quãng đường tích lũy (m), do module Speed & Distance bổ sung.
- **has_ball** — cờ đánh dấu cầu thủ đang giữ bóng tại khung hình hiện tại.

Thiết kế gom mọi thông tin về một cấu trúc duy nhất giúp các module nối tiếp dễ dàng đọc/ghi mà không cần truyền nhiều tham số rời rạc.

3.4.5 Nội suy vị trí bóng

Do quả bóng có kích thước nhỏ, di chuyển nhanh và thường bị che khuất bởi chân cầu thủ, mô hình phát hiện thường xuyên **bỏ sót bóng** ở một số khung hình. Điều này tạo ra các “lỗ hổng” trong quỹ đạo bóng, gây gián đoạn khi tính kiểm soát bóng và vẽ bản đồ.

Để khắc phục, hàm `interpolate_ball_positions` lấp đầy các khung hình thiếu bóng bằng **nội suy tuyến tính** dựa trên các khung hình lân cận có dữ liệu. Cụ thể, chuỗi tọa độ bbox của bóng được đưa vào cấu trúc bảng (DataFrame của pandas) với bốn cột (x_1, y_1, x_2, y_2) , sau đó áp dụng hai phép:

- **Nội suy tuyến tính (interpolate):** với một khung hình thiếu nằm giữa hai khung hình t_1 và t_2 đã biết, giá trị tọa độ được ước lượng theo:

$$p(t) = p(t_1) + \frac{t - t_1}{t_2 - t_1} (p(t_2) - p(t_1))$$

- **Điền lùi (bfill):** với các khung hình thiếu nằm ở đầu chuỗi (chưa có giá trị nào phía trước để nội suy), giá trị được điền bằng giá trị hợp lệ gần nhất phía sau.

Kết quả là một quỹ đạo bóng liên tục, mượt mà qua toàn bộ video.

3.4.6 Thêm điểm vị trí đại diện

Để phục vụ các bước phân tích không gian, mỗi đối tượng cần được quy về một điểm đại diện duy nhất thay vì cả bounding box. Hàm `add_position_to_tracks` thực hiện việc này với quy tắc khác nhau theo loại đối tượng:

- **Bóng:** lấy **tâm bounding box**, vì bóng là vật thể nhỏ gần tròn, tâm là điểm đại diện tự nhiên.

$$\text{position}_{ball} = \left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2} \right)$$

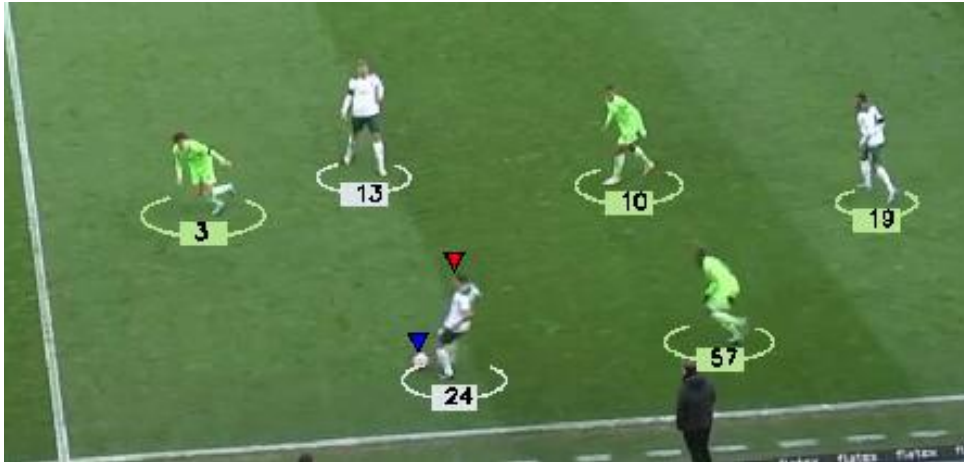
- **Cầu thủ và trọng tài:** lấy **điểm chân** — tâm cạnh dưới của bounding box, vì điểm tiếp đất mới phản ánh đúng vị trí của người trên mặt sân (phần thân trên không nằm trên mặt phẳng sân).

$$\text{position}_{person} = \left(\frac{x_1 + x_2}{2}, y_2 \right)$$

3.4.7 Cơ chế stub

Phát hiện và theo dõi là bước tốn tài nguyên nhất trong pipeline (phải chạy YOLO trên toàn bộ khung hình). Để tránh phải tính toán lại mỗi lần chạy thử, module hỗ trợ cơ chế **stub**: kết quả tracks sau lần chạy đầu được lưu ra file `.pkl` (ví dụ `stubs/track_stubs.pkl`). Ở các lần chạy sau, chỉ cần bật cờ `read_from_stub = True`, hệ thống đọc thẳng kết quả từ file thay vì

chạy lại detection. Cơ chế này đặc biệt hữu ích khi cần gỡ lỗi hoặc tinh chỉnh các module phía sau (gán đội, vẽ chú thích, bản đồ chiến thuật) mà không muốn lặp lại bước nặng nhất.



Hình 3.5: Kết quả module Tracking: cầu thủ và trọng tài được vẽ ellipse kèm track id, quả bóng được đánh dấu bằng tam giác

3.5 Ước lượng chuyển động camera (`camera_movement_estimateor.py`)

Vì camera broadcast liên tục lia và zoom, vị trí pixel của đối tượng bị “trôi” theo camera dù bản thân đối tượng đứng yên. Module này đo độ dịch chuyển của camera giữa các khung hình rồi trừ ngược lại, để vị trí thu được phản ánh đúng chuyển động thực của cầu thủ. Nền tảng lý thuyết là quang luồng Lucas–Kanade đã trình bày ở Chương 2.

3.5.1 Trích đặc trưng từ vùng nền tĩnh

Để đo chuyển động camera một cách đáng tin cậy, cần bám theo những điểm **cố định trên sân** chứ không phải cầu thủ đang chạy. Vấn đề là cầu thủ tập trung chủ yếu ở vùng giữa khung hình, nên module giới hạn việc trích đặc trưng vào **hai dải biên trái và phải** — nơi thường chỉ có khán đài, đường biên, bảng quảng cáo (các thành phần tĩnh).

Cụ thể, một mặt nạ (*mask*) được tạo, chỉ kích hoạt hai dải: cột 0--20 (biên trên) và cột 900--1050 (biên dưới). Trên vùng mặt nạ này, hàm `cv2.goodFeaturesToTrack` chọn ra các điểm góc (*corner features*) tốt nhất để theo dõi (tối đa 100 điểm, `qualityLeve = 0.3`, `minDistance = 3`).



Hình 3.6: Khung hình với overlay hiển thị giá trị Camera movement X và Y

3.5.2 Tính độ dịch chuyển bằng Lucas–Kanade

Với các điểm đặc trưng đã chọn ở khung hình trước, module dùng quang luồng kim tự tháp Lucas–Kanade (`cv2.calcOpticalFlowPyrLK`, cấu hình `winSize = (15,15)`, `maxLevel = 2`) để tìm vị trí mới của chúng ở khung hình hiện tại.

Thay vì lấy trung bình toàn bộ điểm, module chọn **điểm có độ dời lớn nhất** làm đại diện cho chuyển động camera:

$$d_{max} = \max_i \sqrt{(x_i^{new} - x_i^{old})^2 + (y_i^{new} - y_i^{old})^2}$$

Độ dịch chuyển camera $(\Delta x_{cam}, \Delta y_{cam})$ chính là vector dời của điểm này. Cơ chế lấy giá trị lớn nhất giúp bắt được chuyển động rõ rệt của camera, tránh bị các điểm nhiễu (độ dời nhỏ) kéo giá trị về 0.

Chuyển động chỉ được ghi nhận khi d_{max} vượt ngưỡng tối thiểu `min_distance = 5 pixel`; nếu nhỏ hơn, coi như camera đứng yên và độ dịch chuyển khung hình đó là $(0, 0)$. Khi có chuyển động đáng kể, module trích lại đặc trưng mới trên khung hình hiện tại để tiếp tục theo dõi chính xác hơn.

3.5.3 Hiệu chỉnh vị trí đối tượng

Sau khi có độ dịch chuyển camera của từng khung hình, hàm `add_adjust_position_to_tracks` hiệu chỉnh vị trí mọi đối tượng bằng cách trừ đi lượng dời do camera:

$$p_{adjusted} = (x - \Delta x_{cam}, y - \Delta y_{cam})$$

Kết quả được lưu vào trường `position_adjusted` của cấu trúc `tracks`. Đây chính là tọa

độ sẽ được dùng cho bước biến đổi phối cảnh kế tiếp, đảm bảo phép ánh xạ sang sân thực không bị nhiễu bởi chuyển động camera.



Hình 3.7: Khung hình với overlay hiển thị giá trị Camera movement X và Y

3.6 Biến đổi phối cảnh — Homography động (view_transformer.py)

Module này ánh xạ vị trí pixel của đối tượng sang **tọa độ thực trên sân** (đơn vị mét), để mọi phép đo (khoảng cách, tốc độ, vị trí trên bản đồ) được thực hiện trên mặt phẳng sân chứ không phải trên ảnh phối cảnh. Vì camera luôn di chuyển, ma trận Homography được ước lượng **động theo từng khung hình** thay vì cố định một lần. Công thức Homography đã trình bày ở Chương 2.

3.6.1 Cấu hình sân chuẩn

Lớp SoccerPitchConfiguration định nghĩa một sân bóng tiêu chuẩn trong hệ tọa độ thực với kích thước **120m × 70m**, cùng đầy đủ các thông số: vòng cấm (penalty box) 41m × 20.15m, vòng 5m50 (goal box) 18.32m × 5.50m, bán kính vòng tròn giữa sân 9.15m, vị trí chấm phạt đền cách khung thành 11m.

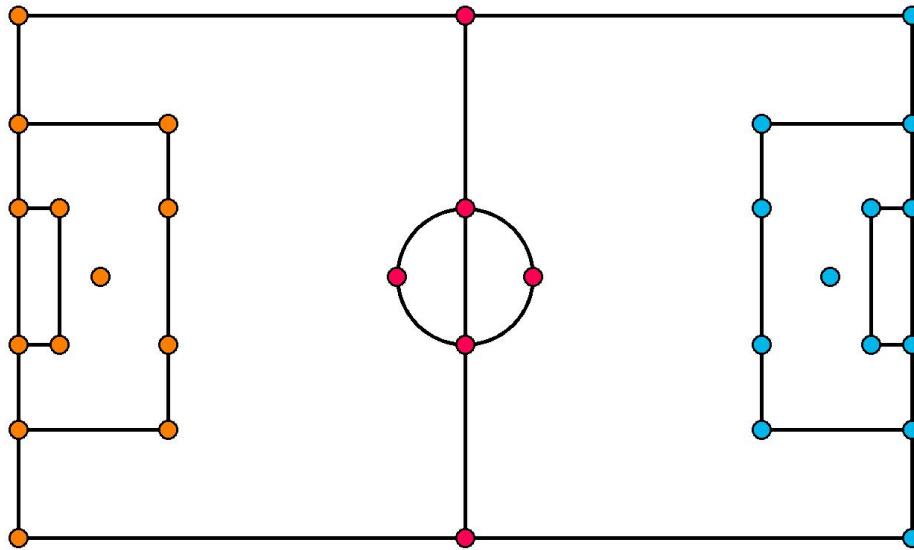
Từ các thông số này, hệ thống tính ra tọa độ thực của **32 điểm mốc (vertices)** đặc trưng phân bố khắp sân — gồm các góc sân, giao điểm vạch vôi với vòng cấm/vòng 5m50, tâm và mép vòng tròn giữa sân... Đây là **tập tọa độ đích đã biết trước**, đóng vai trò “khuôn mẫu” để ghép cặp với các keypoint mà mô hình phát hiện trên ảnh.

3.6.2 Ước lượng ma trận H động theo từng khung hình

Với mỗi khung hình, mô hình YOLO-Pose (best1_pitch.pt) phát hiện các pitch keypoint kèm độ tin cậy. Quy trình ước lượng Homography:

1. Lấy tọa độ pixel các keypoint phát hiện được và độ tin cậy tương ứng.
2. **Lọc theo độ tin cậy:** chỉ giữ những keypoint có $\text{conf} > 0.9$, loại bỏ các điểm phát hiện kém chắc chắn để tránh làm hỏng phép ước lượng.
3. Ghép cặp các keypoint còn lại (tọa độ pixel) với tọa độ thực tương ứng trong bộ 32 vertices.
4. **Yêu cầu tối thiểu 4 điểm:** Homography có 8 bậc tự do nên cần ít nhất 4 cặp điểm để giải. Nếu đủ, gọi `cv2.findHomography` với phương pháp **RANSAC** (ngưỡng tái chiếu 5.0) để ước lượng H bền vững trước các điểm ngoại lai (*outliers*).

RANSAC giúp loại bỏ các cặp keypoint bị lệch (do phát hiện sai vị trí) khỏi quá trình tính toán, cho ra ma trận H ổn định hơn so với giải trực tiếp.



Hình 3.8: Các điểm mốc sân (pitch keypoints) được lấy để tính toán

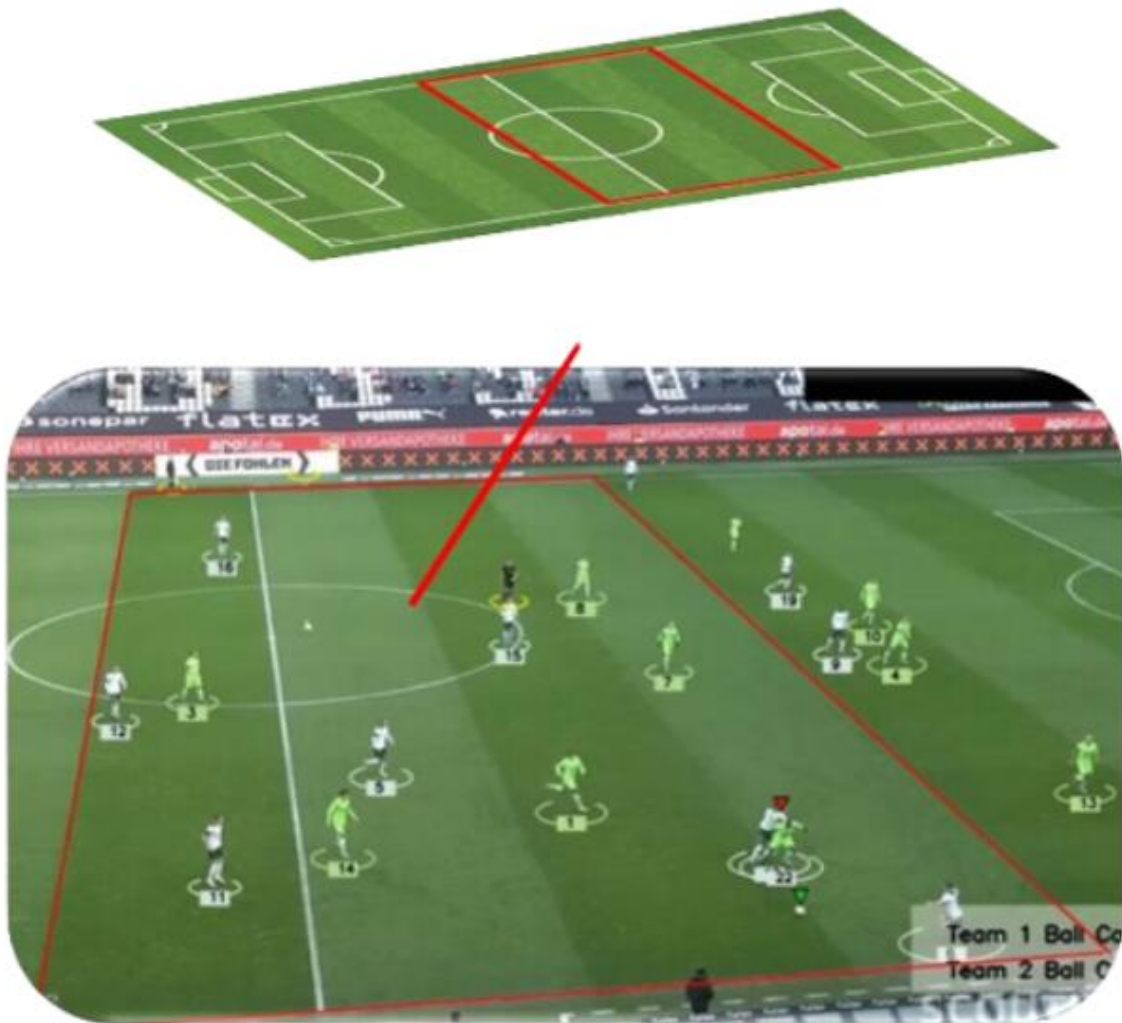
3.6.3 Cơ chế dự phòng (fallback)

Không phải khung hình nào cũng phát hiện đủ keypoint chất lượng (ví dụ camera zoom sát, chỉ thấy một phần sân). Để pipeline không bị gián đoạn, module áp dụng cơ chế dự phòng nhiều lớp:

- Nếu khung hình hiện tại **không ước lượng được H** (thiếu keypoint hoặc dưới 4 điểm), dùng lại **ma trận H hợp lệ gần nhất** (`last_valid_H`) — dựa trên giả định camera thay đổi liên tục nhưng mượt, nên H của khung hình liền trước vẫn xấp xỉ đúng.
- Ở thời điểm khởi tạo (chưa có H hợp lệ nào), dùng một **ma trận H tĩnh mặc định** (`default_H`) được tính sẵn từ một bộ điểm tham chiếu cố định.

3.6.4 Áp dụng phép biến đổi

Với mỗi đối tượng trong tracks, module ưu tiên sử dụng biến `position_adjusted` (đã bù chuyển động camera). Nếu biến này không tồn tại, hệ thống sử dụng `position`. Tọa độ thu được được biến đổi bằng hàm `cv2.perspectiveTransform` với ma trận H của khung hình tương ứng để xác định vị trí thực trên sân và lưu vào trường `position_transformed`. Các điểm nằm ngoài vùng ánh xạ hợp lệ được gán giá trị `None`.



Hình 3.9: Quá trình biến đổi phối cảnh - Lấy 4 điểm rồi tính toán

3.6.5 Cơ chế stub cho Homography

Vì chạy YOLO-Pose trên từng khung hình rất tốn thời gian, toàn bộ chuỗi ma trận $H_{\text{per_frame}}$ được lưu cache ra file `.pkl` (ví dụ `stubs/homography_stubs.pkl`) sau lần chạy đầu. Các lần sau đọc thẳng từ cache, bỏ qua bước ước lượng nặng — tương tự cơ chế stub của module Tracking.

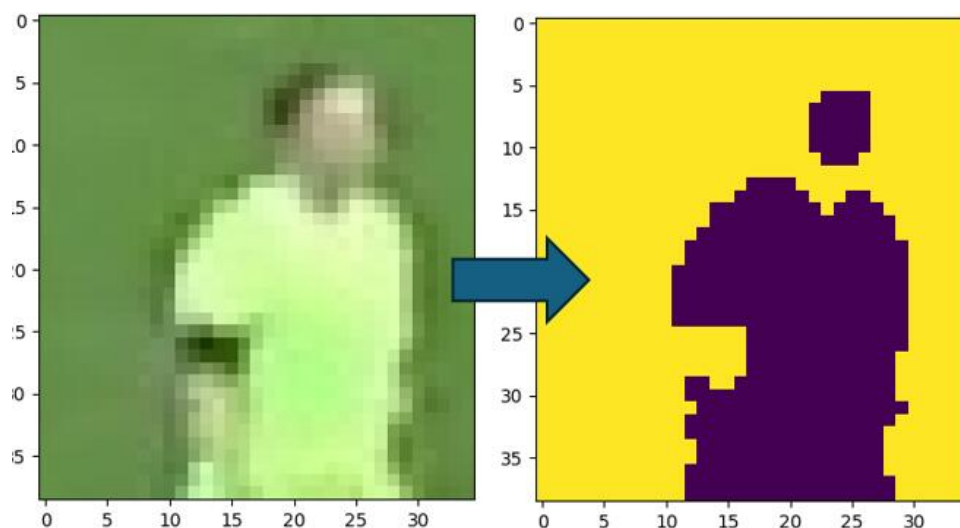
3.7 Gán đội theo màu áo (team_assign.py)

Module này phân loại mỗi cầu thủ vào một trong hai đội dựa trên màu áo đấu, sử dụng thuật toán K-Means (đã trình bày ở Chương 2). Quá trình gồm hai cấp phân cụm: tách màu áo khỏi nền cho từng cầu thủ, rồi gom màu áo của toàn bộ cầu thủ thành hai đội.

3.7.1 Trích xuất màu áo cầu thủ

Để lấy được màu áo của một cầu thủ, module thực hiện:

1. **Cắt vùng áo:** từ bounding box, chỉ lấy **nửa trên** — vùng chứa thân áo, loại bỏ phần quần và chân (vốn dễ lẫn màu với sân cỏ).
2. **Phân cụm tách áo/nền:** chạy K-Means với $k = 2$ trên các pixel của vùng nửa trên, chia thành hai cụm màu. Một cụm là màu áo, cụm còn lại là màu nền (cỏ, khán giả lọt vào khung).
3. **Xác định cụm nền:** dựa vào giả định viền bbox thường là nền, module xét nhãn cụm tại **bốn góc** của vùng ảnh. Cụm xuất hiện nhiều nhất ở các góc được coi là **nền**; cụm còn lại là **áo cầu thủ**.
4. Tâm (centroid) của cụm áo chính là **màu áo đại diện** của cầu thủ đó.



Hình 3.10: Dùng kmean để trích xuất đặc trưng

3.7.2 Phân hai đội

Sau khi có màu áo của từng cầu thủ, module gom **màu áo của tất cả cầu thủ trong khung hình đầu** thành một tập, rồi chạy K-Means lần thứ hai (cũng với $k = 2$) trên tập màu này. Hai tâm cụm thu được chính là **màu đại diện của hai đội**, lưu vào `team_colors[1]` và `team_colors[2]`. Bộ phân cụm (kmeans) này được giữ lại để gán đội cho các cầu thủ ở những khung hình sau.

3.7.3 Gán đội và lưu cache

Với mỗi cầu thủ mới, module trích màu áo rồi dùng bộ K-Means đã huấn luyện để dự đoán cầu thủ thuộc cụm/đội nào (`team_id` được quy về 1 hoặc 2).

Để tránh tính toán lặp lại, kết quả được lưu trong từ điển `player_team_dict` ánh xạ vào team. Khi gặp lại một cầu thủ đã được gán đội trước đó, module trả về kết quả từ cache ngay lập tức thay vì trích màu áo và phân cụm lại — vừa tăng tốc, vừa đảm bảo một cầu thủ giữ nguyên đội xuyên suốt video.



Hình 3.11: Kết quả gán đội: cầu thủ hai đội được tô bằng hai màu khác nhau

3.8 Module ReID — 5 tầng (`reid.py`)

ByteTrack chỉ duy trì định danh trong các phiên ngắn; khi cầu thủ bị che khuất lâu, chạy ra/vào khung hình hoặc tranh chấp đồng người, định danh dễ bị đặt lại (*ID switch*). Module ReID giải quyết bằng cách nhận dạng lại cùng một cầu thủ dựa trên đặc trưng ngoại hình, kết hợp nhiều ràng buộc đặc thù của bóng đá. Điểm cốt lõi của module là cơ chế so khớp **5 tầng** xếp theo thứ tự ưu tiên.

3.8.1 Trích đặc trưng OSNet

Hàm `extract_feature` trích vector đặc trưng ngoại hình của mỗi cầu thủ qua mạng OSNet:

1. Cắt vùng ảnh trong bounding box. Nếu bbox **quá nhỏ** (rộng dưới 20px hoặc cao dưới 40px), bỏ qua — vì ảnh quá nhỏ cho đặc trưng kém tin cậy.
2. Chuyển hệ màu BGR → RGB, resize về kích thước chuẩn **256 × 128**, chuẩn hóa theo giá trị trung bình và độ lệch chuẩn của ImageNet.
3. Đưa qua OSNet để thu vector embedding, sau đó **chuẩn hóa L2** (đưa về vector đơn vị) để phép đo cosine similarity về sau chính xác.

3.8.2 Kiến trúc gallery

Mỗi cầu thủ (định danh toàn cục) được lưu trong một “thư viện” (*gallery*) chứa toàn bộ thông tin cần cho việc so khớp:

```
gallery[global_id] = {
    "features":      [...]    # buffer tối đa 10 embedding gần nhất
    "last_pos_px":   (x, y)    # vị trí pixel cuối cùng
    "last_pos_m":    (x, y)    # vị trí thực (mét) cuối cùng
    "last_frame":    int       # khung hình cuối thấy
    "team":          int       # đội
    "zone_counts":   {...}     # số lần xuất hiện ở mỗi vùng sân
    "primary_zone":  str       # vùng hoạt động chính
}
```

Việc lưu nhiều embedding (thay vì một) giúp đại diện ngoại hình cầu thủ ổn định hơn trước thay đổi góc nhìn và ánh sáng; còn vị trí pixel/mét, khung hình cuối và đội là cơ sở cho các tầng ràng buộc bên dưới.

3.8.3 Năm tầng so khớp

Khi một cầu thủ cần được khớp với gallery, hàm `_match_with_gallery` xét lần lượt qua năm tầng theo thứ tự ưu tiên. Tầng trước có quyền quyết định hoặc loại bỏ ứng viên trước khi xét tầng sau.

Tầng 1 — Jacket logic (vị trí + thời gian). Nếu một định danh trong gallery có vị trí pixel rất gần cầu thủ hiện tại (dưới `jacket_dist_thresh`, khoảng 60px) **và** mới mất dấu trong thời gian ngắn (dưới `jacket_time_thresh` khung hình), hệ thống khẳng định ngay đây là cùng một người, **không cần so sánh đặc trưng**. Tầng này xử lý hiệu quả các trường hợp che khuất chớp nhoáng — nhanh và gần như chắc chắn đúng.

Tầng 2 — Lọc theo đội. Nếu ứng viên trong gallery thuộc **đội khác** với cầu thủ hiện tại, loại bỏ ngay. Hai cầu thủ khác đội không thể là cùng một người, nên ràng buộc này cắt giảm đáng kể không gian tìm kiếm và tránh nhầm lẫn chéo đội.

Tầng 3 — Ràng buộc vật lý. Một cầu thủ không thể “dịch chuyển tức thời” trên sân. Dựa trên tốc độ tối đa của con người (`MAX_PLAYER_SPEED_MS` = 11 m/s, ≈ 40 km/h) và thời gian đã trôi qua, hệ thống tính quãng đường tối đa khả dĩ rồi so với khoảng cách thực tế:

$$\Delta t = \frac{\text{time_gap}}{\text{frame_rate}}, \quad d_{max} = v_{max} \cdot \Delta t$$

Nếu khoảng cách thực tế giữa hai vị trí (đơn vị mét) vượt quá d_{max} cộng thêm 2m biên độ sai số:

$$d_{thc} > v_{max} \cdot \Delta t + 2$$

thì ứng viên bị loại vì vi phạm quy luật vật lý — không thể nào là cùng một người.

Tầng 4 — OSNet + ngưỡng thích nghi. Với các ứng viên còn lại, hệ thống tính cosine similarity giữa embedding hiện tại và các embedding trong gallery, lấy giá trị lớn nhất làm điểm khớp. Ngưỡng chấp nhận **tăng dần theo thời gian mất dấu** (`time_gap`), vì mất dấu càng lâu thì rủi ro nhầm lẫn càng cao nên cần ngưỡng chặt hơn:

- `time_gap < 72` khung hình (≈ 3 giây): ngưỡng 0.75.
- `72 \leq time_gap < 720` ($\approx 3-30$ giây): ngưỡng 0.80.
- `time_gap \geq 720` (> 30 giây): ngưỡng 0.85.

Những ứng viên có điểm khớp vượt ngưỡng tương ứng được đưa vào danh sách ứng viên hợp lệ.

Tầng 5 — Formation prior (tie-break theo vùng). Nếu chỉ có một ứng viên, chọn luôn. Nếu có nhiều ứng viên với điểm khớp **sát nhau** (chênh lệch dưới 0.03), hệ thống dùng thông tin **vùng hoạt động** làm yếu tố quyết định: sân được chia thành lưới 3×2 (trái/giữa/phải $\times$ phòng thủ/tấn công), gồm 6 vùng. Ứng viên có vùng hoạt động chính (`primary_zone`) trùng với vùng hiện tại của cầu thủ sẽ được ưu tiên. Đây là tri thức tiên nghiệm về đội hình: mỗi cầu thủ thường hoạt động trong một khu vực cố định, nên vị trí giúp phân biệt hai cầu thủ có ngoại hình tương tự.

3.8.4 Cập nhật gallery

Sau khi khớp xong, hàm `_update_gallery` cập nhật thông tin cho định danh tương ứng:

- **Rolling buffer 10 embedding:** thêm embedding mới vào buffer, giữ tối đa 10 mẫu gần nhất. Buffer này giúp đặc trưng cầu thủ thích nghi dần với thay đổi ngoại hình mà vẫn ổn định.
- Cập nhật vị trí pixel/mét, khung hình cuối, đội.
- **Cập nhật vùng:** tăng bộ đếm `zone_counts` cho vùng hiện tại, rồi đặt `primary_zone` là vùng có số lần xuất hiện nhiều nhất.

3.8.5 Hợp nhất offline, ánh xạ ID và stub

Hàm `merge_tracks_offline` chạy ReID trên toàn bộ video theo chế độ offline:

- Duyệt qua từng khung hình, với mỗi định danh cũ (do ByteTrack cấp), trích đặc trưng và gọi `_match_with_gallery` để tìm định danh toàn cục.
- Một từ điển `id_map` ánh xạ định danh cũ $\rightarrow$ định danh toàn cục: nếu một định danh cũ đã từng được khớp, các lần sau dùng lại kết quả mà không khớp lại. Nếu không tìm được định danh phù hợp, giữ nguyên định danh cũ làm định danh mới.

- Kết quả là cấu trúc `tracks` với định danh cầu thủ đã được hợp nhất ổn định; nhóm trọng tài và bóng giữ nguyên.
- Toàn bộ kết quả được lưu **stub** ra file `.pkl` để các lần chạy sau bỏ qua bước ReID nặng.

3.9 Tốc độ và quãng đường (`speed_and_distance_estimator.py`)

Sau khi vị trí cầu thủ đã được ánh xạ sang tọa độ sân thực (mét), module này tính tốc độ và quãng đường di chuyển. Cơ sở lý thuyết (công thức động học) đã trình bày ở Chương 2.

Module xử lý theo **cửa sổ 5 khung hình** (`frame_window = 5`) với tốc độ khung hình `frame_rate = 24`. Nhóm **bóng và trọng tài được bỏ qua** vì không cần thống kê thể lực cho hai đối tượng này.

Trong mỗi cửa sổ, với một cầu thủ xuất hiện ở cả khung hình đầu và khung hình cuối, module tính:

- **Quãng đường** giữa hai vị trí thực:

$$d = \sqrt{(x_{end} - x_{start})^2 + (y_{end} - y_{start})^2}$$

- **Thời gian trôi qua và tốc độ:**

$$\Delta t = \frac{\text{số khung hình}}{\text{frame_rate}}, \quad v_{km/h} = \frac{d}{\Delta t} \times 3.6$$

- **Quãng đường tích lũy:** cộng dồn d vào `total_distance` của cầu thủ.

Tốc độ và quãng đường tích lũy được ghi vào các trường `speed`, `distance` của tất cả khung hình trong cửa sổ. Nếu vị trí thực của cầu thủ là `None` (nằm ngoài vùng ánh xạ hợp lệ của Homography), cửa sổ đó được **bỏ qua** để tránh tính toán sai.



Hình 3.12: Khung hình hiển thị tốc độ (km/h) và quãng đường (m) dưới chân mỗi cầu thủ

3.10 Kiểm soát bóng (player_ball_assign.py và logic trong main.py)

Module xác định cầu thủ đang giữ bóng và từ đó tính tỷ lệ kiểm soát bóng của hai đội.

Gán bóng cho cầu thủ. Với mỗi cầu thủ, hệ thống tính khoảng cách từ tâm bóng tới hai điểm chân (góc dưới trái và góc dưới phải của bounding box) rồi lấy giá trị nhỏ hơn:

$$d_{player} = \min(\text{dist}(p_{ball}, p_{left}), \text{dist}(p_{ball}, p_{right}))$$

Cầu thủ có d_{player} nhỏ nhất và nằm dưới ngưỡng $\text{max_player_ball_distance} = 50$ pixel được xác định là người giữ bóng (đánh dấu $\text{has_ball} = \text{True}$). Nếu không cầu thủ nào thỏa ngưỡng, khung hình đó không gán ai giữ bóng.

Logic kiểm soát bóng theo đội. Tại mỗi khung hình, đội của cầu thủ giữ bóng được ghi vào danh sách team_ball_control . Khi không xác định được người giữ bóng, hệ thống **kế thừa kết quả khung hình trước** (giả định bóng vẫn thuộc quyền kiểm soát của đội đang giữ trước đó); nếu chưa có lịch sử nào, gán mặc định cho đội 2.

Tỷ lệ kiểm soát bóng của một đội tính đến khung hình hiện tại:

$$\text{Possession}_{team} = \frac{N_{team}}{N_{team_1} + N_{team_2}} \times 100\%$$

trong đó N_{team} là số khung hình đội đó được ghi nhận giữ bóng.



Hình 3.13: Khung hình hiển thị ô tỷ lệ kiểm soát bóng của hai đội

3.11 Trực quan hóa: Tactical Map & Pitch Renderer

Hai module này dựng bản đồ chiến thuật 2D top-down và overlay lên video, cho cái nhìn tổng quan về vị trí và di chuyển của toàn đội.

3.11.1 Render sân 2D (PitchRenderer)

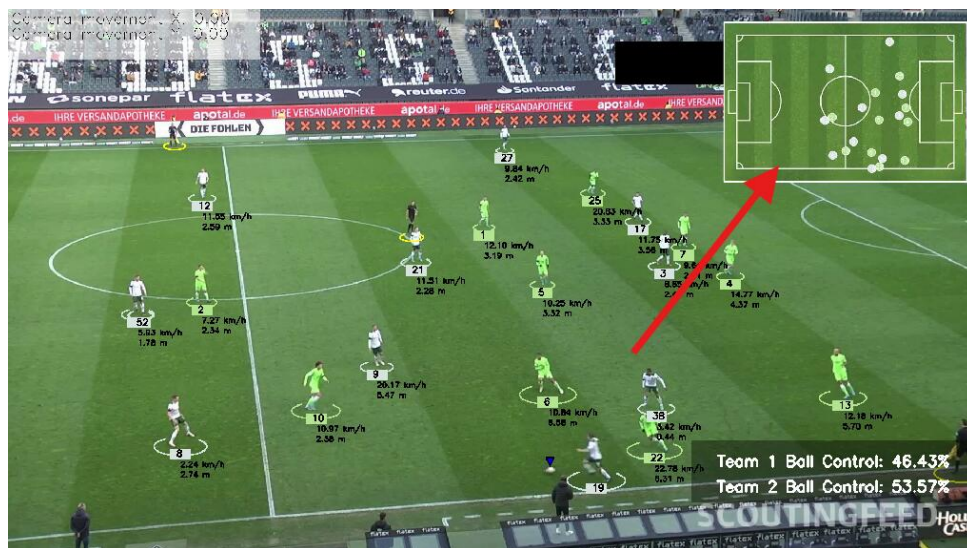
Lớp PitchRenderer vẽ một sân bóng 2D theo phong cách tối giản. Từ kích thước sân thực ($120m \times 70m$) và kích thước ảnh đích, module tính tỷ lệ **mét** $\rightarrow$ **pixel** rồi vẽ đầy đủ các chi tiết: đường biên ngoài, đường giữa sân, vòng tròn giữa và chấm giữa, hai vòng cấm, hai vòng 5m50, hai chấm phạt đền, hai cung phạt đền và bốn cung phạt góc. Mỗi chi tiết được vẽ đúng tỷ lệ theo kích thước thực, tạo ra một sân chuẩn để định vị cầu thủ.

3.11.2 Tactical Map (TacticalMap)

Module TacticalMap vẽ vị trí đối tượng lên bản đồ rồi chèn (Picture-in-Picture) vào **góc trên phải** khung hình video:

- **Cầu thủ:** mỗi cầu thủ là một chấm tròn tô theo `team_color`, viền trắng để tương phản.
- Vị trí thực (mét) của mỗi đối tượng được quy đổi sang pixel trên bản đồ qua hàm `meter_to_pixel`, có giới hạn (`clamp`) để không vẽ tràn ra ngoài bản đồ.

Bản đồ được viền bằng khung trắng và overlay lên video, tạo thành một “thẻ” gọn gàng ở góc khung hình.



Hình 3.14: Bản đồ chiến thuật 2D

3.12 Tổng hợp thống kê và sinh báo cáo (`stats_aggregator`, `nlp_report`)

Sau khi pipeline chính hoàn tất, hai module `stats_aggregator` và `nlp_report` đảm nhận khâu cuối: tổng hợp toàn bộ dữ liệu đã trích xuất thành thống kê cấp cầu thủ/đội, sinh bản đồ nhiệt, và tạo báo cáo chiến thuật tự động bằng mô hình ngôn ngữ lớn (LLM).

3.12.1 Tổng hợp thống kê (`stats_aggregator`)

Lớp `StatsExporter` điều phối toàn bộ quá trình phân tích thông qua hàm `process_and_export`, lần lượt gọi bốn thành phần con:

a) Xác định hướng tấn công. Trước tiên, hàm `determine_team_directions` tính tọa độ X trung bình của mỗi đội trên sân thực. Đội có tọa độ X trung bình nhỏ hơn được xác định phòng thủ ở nửa sân trái ($x = 0$) và tấn công sang phải ($x = 120$), và ngược lại. Hướng tấn công này là cơ sở để chuẩn hóa vị trí khi tính vùng hoạt động và đội hình.

b) Thống kê cầu thủ (`PlayerStatsAggregator`). Với mỗi cầu thủ, module tổng hợp từ dữ liệu `tracks`: tổng quãng đường (mét), tốc độ trung bình và tốc độ tối đa (km/h), thời gian/số khung hình giữ bóng, vùng hoạt động ưa thích và phân bố theo vùng (*zone distribution*). Vùng của mỗi vị trí được xác định tương đối theo hướng tấn công của đội (hàm `get_relative_zone`).

c) Thống kê đội (`TeamStatsAggregator`). Ở cấp độ đội, module tính các chỉ số chiến thuật:

- **Tỷ lệ kiểm soát bóng:** từ mảng `team_ball_control`.
- **Tổng quãng đường và tốc độ trung bình:** tổng hợp từ thống kê cầu thủ.

- **Độ nén đội hình (compactness):** đo bằng diện tích bao lồi (*convex hull area*) và độ phân tán trung bình của cầu thủ quanh trọng tâm đội (*average spread*) tại mỗi khung hình.
- **Cường độ pressing:** với mỗi khung hình mà đối phương giữ bóng, module tính khoảng cách từ hậu vệ gần nhất tới cầu thủ cầm bóng; nếu khoảng cách $\leq 5\text{m}$ thì tính là một khung hình pressing. Cường độ pressing là tỷ lệ phần trăm số khung hình pressing trên tổng số khung hình đối phương cầm bóng.

d) Phát hiện đội hình (FormationDetector). Module ước lượng sơ đồ chiến thuật của mỗi đội. Trước tiên, thủ môn được xác định là cầu thủ có tọa độ X trung bình gần khung thành nhà nhất và được loại khỏi tính toán. Các cầu thủ còn lại được phân cụm theo tọa độ X (đã chuẩn hóa theo hướng tấn công) thành **ba tuyến** (hậu vệ, tiền vệ, tiền đạo) bằng thuật toán **K-Means 1 chiều** tự cài đặt với $k = 3$. Số cầu thủ ở mỗi tuyến tạo thành chuỗi sơ đồ dạng “hậu vệ–tiền vệ–tiền đạo” (ví dụ 4-3-3). Đội hình được phát hiện cả ở mức toàn trận và theo từng cửa sổ 30 giây để theo dõi biến động chiến thuật.

e) Sinh bản đồ nhiệt (HeatmapGenerator). Module dựng heatmap cho từng cầu thủ và từng đội. Vị trí thực (mét) được tích lũy vào một ma trận, làm mờ bằng **Gaussian Blur** (kernel 51) để tạo vùng nhiệt mượt, chuẩn hóa về dải 0–255, áp bảng màu **JET**, rồi pha trộn (alpha blending tối đa 0.65) lên nền sân 2D vẽ sẵn.

Toàn bộ kết quả được gom vào một từ điển và lưu ra file **JSON** (`match_stats.json`) gồm: tóm tắt trận đấu, hướng tấn công, thống kê hai đội và danh sách thống kê từng cầu thủ.

3.12.2 Sinh báo cáo tự động bằng LLM (`nlp_report`)

Module `nlp_report` (chạy riêng qua `main_report.py`) đọc file `match_stats.json` và sinh báo cáo chiến thuật dạng PDF qua ba bước:

a) Chuẩn hóa và dựng prompt. Hàm `adapt_stats` chuyển đổi định dạng JSON gốc sang cấu trúc mà module báo cáo yêu cầu. Sau đó `PromptBuilder` dựng cặp *system prompt* (định nghĩa vai trò “chuyên gia phân tích chiến thuật bóng đá” và yêu cầu trả về đúng cấu trúc JSON) và *user prompt* (bảng số liệu trận đấu: kiểm soát bóng, đội hình, quãng đường, tốc độ, độ nén, pressing, phân bố vùng, top cầu thủ theo quãng đường và số lần chạm bóng).

b) Gọi mô hình ngôn ngữ. Lớp `LLMClient` hỗ trợ nhiều nhà cung cấp (Gemini, Claude, OpenAI, Groq), tự lấy API key từ biến môi trường. Client có cơ chế **thử lại (retry)** với thời gian chờ tăng dần và **chuyển mô hình dự phòng** khi mô hình chính quá tải (lỗi 503). Phản hồi được làm sạch (loại bỏ markdown), phân tích thành JSON và kiểm tra đủ các khóa bắt buộc (tổng quan trận, phân tích từng đội, so sánh, kết luận).

c) Dựng file PDF. Cuối cùng, `PDFBuilder` dựng báo cáo PDF hoàn chỉnh từ nội dung LLM sinh ra kết hợp các số liệu thống kê (và heatmap), tạo ra tài liệu phân tích chiến thuật trực quan cho người dùng.

4 Chương 4: Thực Nghiệm Và Đánh Giá

4.1 Mô tả tập dữ liệu

4.1.1 Thu thập dữ liệu

Dữ liệu được xây dựng từ các video bóng đá broadcast thực tế. Các khung hình được tách ra và gán nhãn thủ công trên nền tảng **Roboflow** với bốn lớp đối tượng: **ball**, **goalkeeper**, **player**, **referee**.

4.1.2 Thống kê tập dữ liệu

Tập dữ liệu gồm **372 ảnh**, chia theo tỉ lệ chuẩn như Bảng 2.

Bảng 2: Phân chia tập dữ liệu

Tập dữ liệu	Tỉ lệ	Số ảnh
Train	80%	298
Validation	13%	49
Test	7%	25
Tổng	100%	372

Phân bố số đối tượng (instance) được gán nhãn theo từng lớp như Bảng 3.

Bảng 3: Phân bố số instance theo lớp

Lớp	Số instance
ball	258
goalkeeper	230
player	5.955
referee	690
Tổng	7.133

Tập dữ liệu **mất cân bằng nghiêm trọng** giữa các lớp: lớp player chiếm áp đảo (5.955 instance, $\approx 83\%$), trong khi ball và goalkeeper chỉ có vài trăm instance. Sự mất cân bằng này cộng với việc quả bóng có kích thước rất nhỏ là nguyên nhân trực tiếp khiến mô hình phát hiện bóng kém hơn hẳn các lớp còn lại (phân tích chi tiết ở mục 4.4.4). Biểu đồ phân bố kích thước hộp bao cho thấy đa số đối tượng có kích thước nhỏ (chiều rộng tập trung quanh 0.01, chiều cao 0.04–0.06 trên ảnh chuẩn hóa), phản ánh đặc thù góc quay broadcast.

4.1.3 Tiền xử lý

- **Auto-Orient:** Tự động chỉnh hướng ảnh theo metadata EXIF.
- **Resize:** Co giãn (stretch) về kích thước chuẩn 640×640 pixel.
- **Tăng cường dữ liệu:** Không áp dụng.

4.2 Dữ liệu thực nghiệm

Hệ thống được kiểm thử trên video trận đấu với điều kiện như Bảng 4.

Bảng 4: Thông số môi trường thực nghiệm

Tham số	Giá trị
Độ phân giải	1920×1080 px
Tốc độ video	25 FPS
Phần cứng thử nghiệm	CPU (không GPU)
Mô hình Detection	YOLOv11s (best_yolo11s.pt)
Mô hình Keypoint sân	YOLOv11-Pose (best1_pitch.pt)
Mô hình ReID	OSNet x1_0 (pretrained)
Tracker	ByteTrack (Supervision)

Các tình huống kiểm thử bao gồm: di chuyển tự do, tranh chấp/che khuất, camera lia nhanh theo bóng.

4.3 Độ đo đánh giá

4.3.1 Độ chính xác phát hiện

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{mAP@0.5} = \text{Mean Average Precision tại ngưỡng IoU} = 0.5$$

$$\text{mAP@0.5:0.95} = \text{Trung bình mAP trên các ngưỡng IoU từ 0.5 đến 0.95}$$

Trong đó TP (*True Positive*) là số đối tượng phát hiện đúng, FP (*False Positive*) là số phát hiện sai (nền bị nhận nhầm thành đối tượng), FN (*False Negative*) là số đối tượng bị bỏ sót.

4.3.2 Confusion Matrix

Ma trận nhầm lẫn thể hiện số lượng dự đoán đúng/sai giữa bốn lớp (ball, goalkeeper, player, referee) và nền (background), giúp xác định lớp nào hay bị bỏ sót hoặc nhầm lẫn.

4.3.3 Độ ổn định Tracking và ReID

- **ID Switch:** số lần cùng một đối tượng bị gán định danh khác nhau. Càng thấp càng tốt.
- **MOTA** (*Multi-Object Tracking Accuracy*): độ đo tổng hợp dựa trên FN, FP và ID Switch.

$$\text{MOTA} = 1 - \frac{\sum_t (\text{FN}_t + \text{FP}_t + \text{IDS}_t)}{\sum_t \text{GT}_t}$$

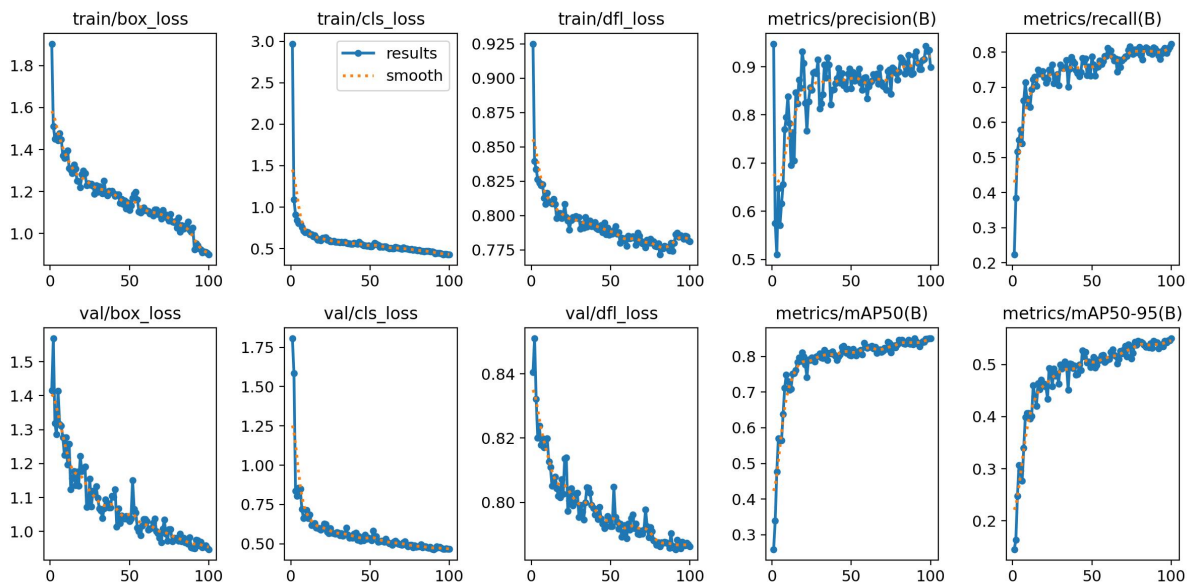
Trong đó FN = False Negative, FP = False Positive, IDS = ID Switch, GT = Ground Truth.

4.4 Kết quả thực nghiệm Detection

4.4.1 Quá trình huấn luyện

Mô hình được huấn luyện trong 100 epoch. Đường cong huấn luyện cho thấy:

- **Box loss** (train) giảm đều từ ≈ 1.9 xuống ≈ 0.9 ; **Cls loss** giảm mạnh từ ≈ 3.0 xuống ≈ 0.45 ; **DFL loss** giảm từ ≈ 0.925 xuống ≈ 0.78 .
- Trên tập validation, cả ba loss đều hội tụ ổn định, không có dấu hiệu overfitting rõ rệt.
- **Precision** ổn định quanh ≈ 0.90 ; **Recall** ≈ 0.81 ; **mAP@0.5** ≈ 0.85 ; **mAP@0.5:0.95** ≈ 0.55 ở epoch cuối.



Hình 4.1: Đường cong Loss và Metrics trong quá trình huấn luyện

4.4.2 Kết quả tổng hợp

Bảng 5: Độ chính xác trung bình theo lớp (AP@0.5)

Lớp	AP@0.5	Nhận xét
player	0.991	Tốt nhất — dữ liệu nhiều, đặc trưng rõ
referee	0.962	Tốt
goalkeeper	0.958	Tốt
ball	0.489	Yếu — vật thể nhỏ, dữ liệu ít
Tất cả (mAP@0.5)	0.850	

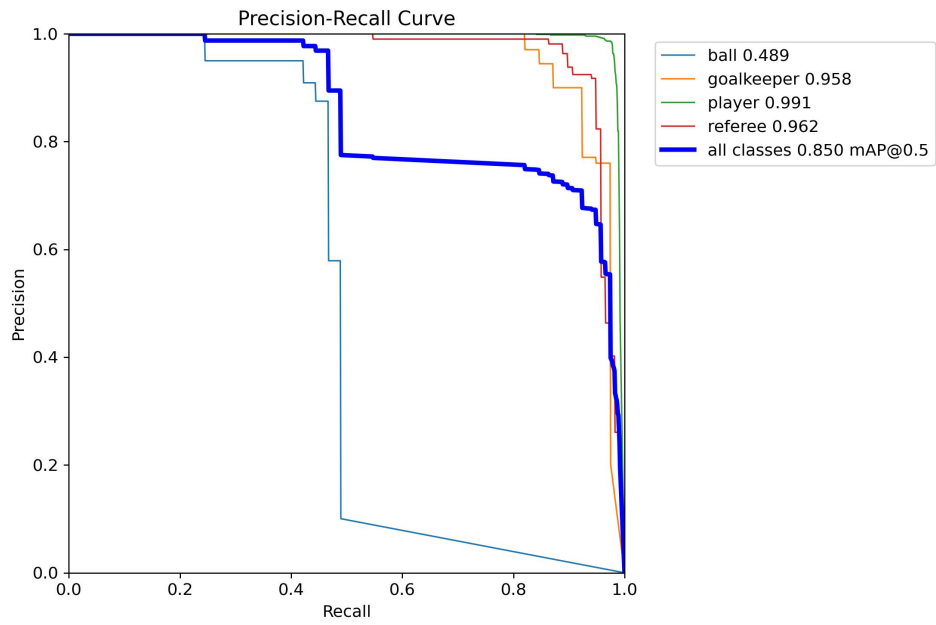
Bảng 6: Các chỉ số tổng quát

Độ đo	Giá trị
mAP@0.5	0.850
mAP@0.5:0.95	≈ 0.55
F1 cực đại	0.85 (tại confidence = 0.295)
Precision cực đại	1.00 (tại confidence = 0.811)
Recall cực đại	0.86 (tại confidence thấp)

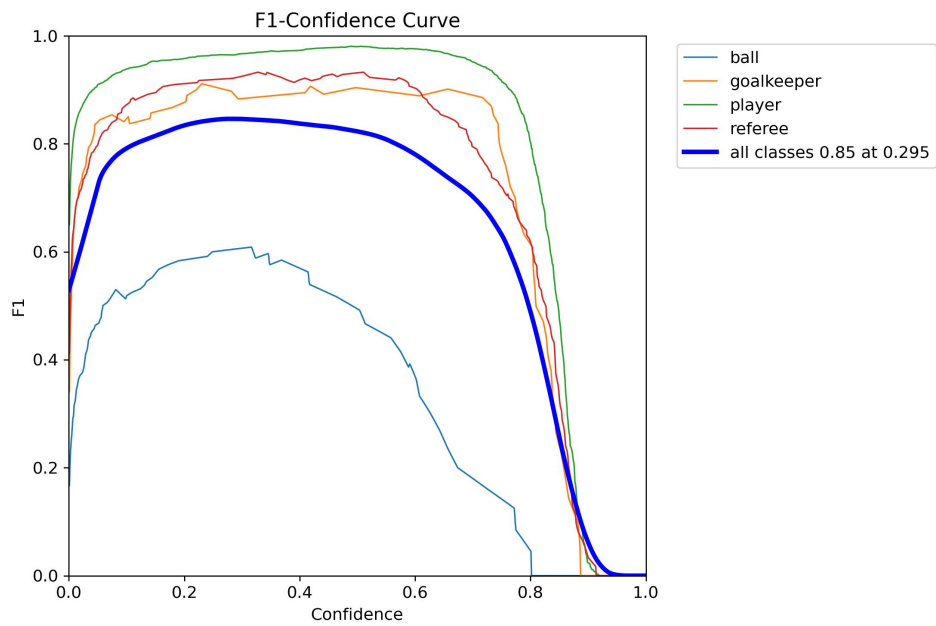
Mô hình đạt $\text{mAP@0.5} = 0.850$, mức tốt cho bài toán bốn lớp. Ba lớp player/referee/goalkeeper đều đạt $\text{AP} > 0.95$, nhưng lớp **ball** kéo **điểm trung bình xuống** (AP chỉ 0.489).

4.4.3 Phân tích các đường cong

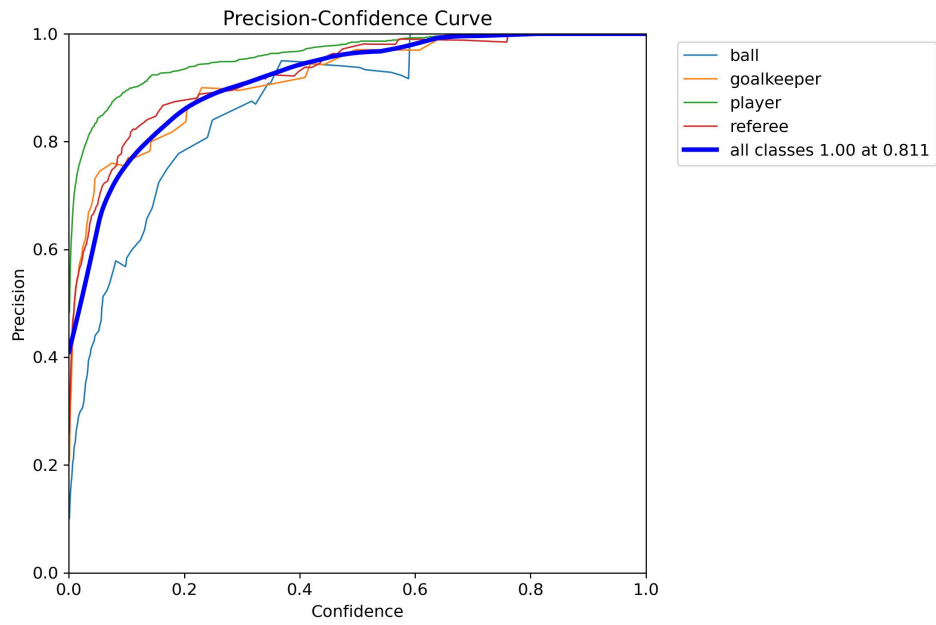
- **Precision–Recall Curve:** ba lớp người/trọng tài gần như vuông góc lý tưởng ($\text{AP} > 0.95$), riêng lớp ball tụt rõ rệt ($\text{AP} = 0.489$).
- **F1–Confidence Curve:** F1 cực đại 0.85 tại confidence 0.295; lớp player duy trì ≈ 0.98 , ball chỉ đạt đỉnh ≈ 0.60 .
- **Precision–Confidence Curve:** precision tiến tới 1.00 khi confidence cao (≈ 0.81).
- **Recall–Confidence Curve:** recall toàn cục đạt 0.86 ở ngưỡng thấp; đường của ball thấp hơn hẳn các lớp khác.



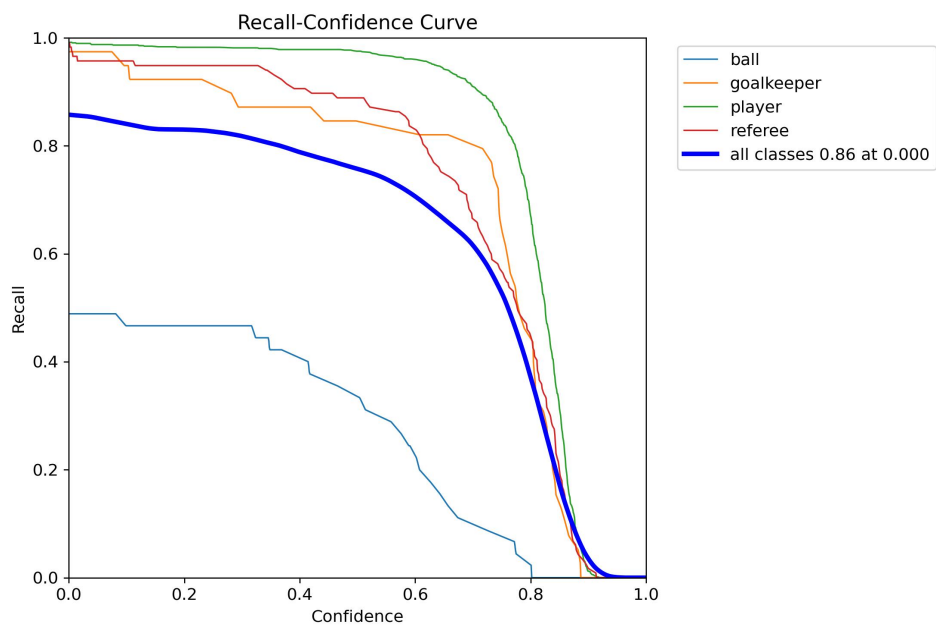
Hình 4.2: Precision-Recall Curve — $\text{mAP}@0.5 = 0.850$; ba lớp người/trọng tài $\text{AP} > 0.95$, lớp ball chỉ đạt 0.489



Hình 4.3: F1-Confidence Curve — F1 cực đại 0.85 tại confidence 0.295



Hình 4.4: Precision-Confidence Curve — precision đạt 1.00 tại confidence ≈ 0.811



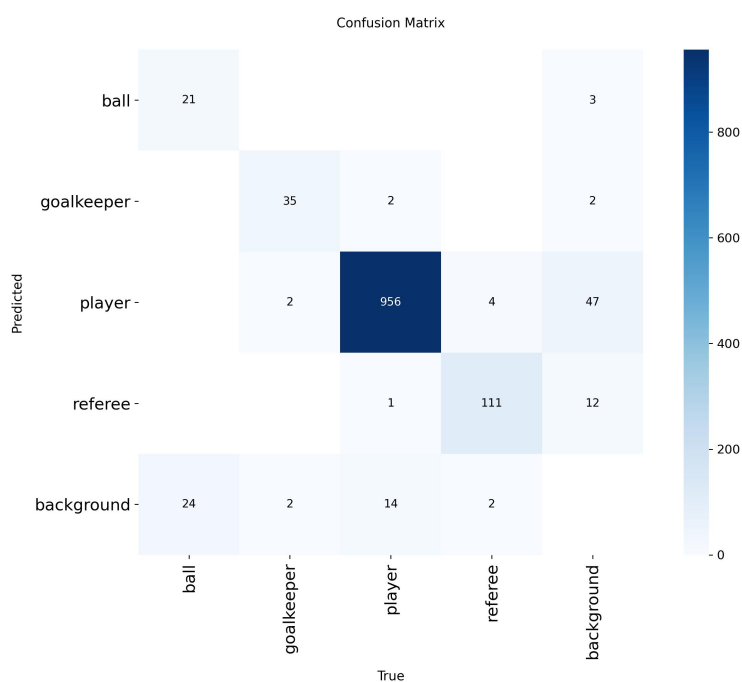
Hình 4.5: Recall-Confidence Curve — recall toàn cục 0.86 ở ngưỡng thấp; đường của ball thấp hơn hẳn

4.4.4 Phân tích Confusion Matrix

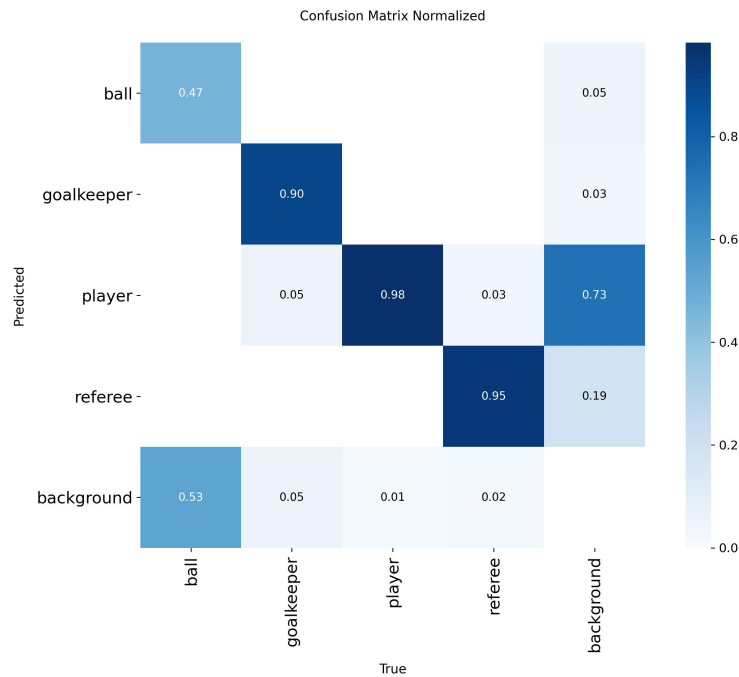
Bảng 7: Tỷ lệ phát hiện đúng theo lớp (từ Confusion Matrix)

Lớp	Phát hiện đúng	Tỷ lệ	Nhận xét
player	956 / 973	0.98	Rất tốt
referee	111 / 117	0.95	Tốt
goalkeeper	35 / 39	0.90	Tốt
ball	21 / 45	0.47	Kém — 24 quả bóng bị bỏ sót (nhận nhầm thành nền)

Phân tích: lớp player phát hiện gần như tuyệt đối (98%). Lớp ball là điểm yếu rõ rệt — chỉ 47% bóng được phát hiện đúng, **24 quả bóng bị bỏ sót**. Đây chính là lý do hệ thống **bắt buộc phải nội suy quỹ đạo bóng** (mục 3.4); kết quả thực nghiệm xác nhận tính cần thiết của cơ chế này. Mô hình cũng sinh một số dương tính giả từ nền (47 nền → player, 12 nền → referee), do khán đài/đám đông phía sau gây nhiễu.



Hình 4.6: Confusion Matrix trên tập Validation



Hình 4.7: Confusion Matrix chuẩn hóa

4.5 Đánh giá Tracking và Re-Identification

Do hệ thống chưa có bộ dữ liệu ground-truth tracking để tính MOTA định lượng, phần này đánh giá **định tính** dựa trên quan sát kết quả thực tế trên video `best.avi` (1920×1080 , 25 FPS).

Hiện tượng ID Switch còn nhiều. Quan sát đoạn đầu video cho thấy chỉ trong khoảng mười giây, số định danh đã tăng vọt: xuất hiện các ID lớn như 155, 188, 216, trong khi số đối tượng thực tế trên sân chỉ khoảng hơn hai mươi. Đặc biệt, nhóm cầu thủ/trọng tài ở biên trái bị gán ID mới liên tục qua các thời điểm ($64 \rightarrow 155 \rightarrow 188 \rightarrow 216$), cho thấy cùng một người bị tái định danh nhiều lần. Nguyên nhân chính:

- Cầu thủ cùng đội mặc áo gần như giống hệt nhau, khiến đặc trưng OSNet khó phân biệt.
- Che khuất và tranh chấp đông người làm đứt quỹ đạo.
- Camera lia mạnh khiến vị trí nhảy, làm các tầng ràng buộc theo vị trí của ReID khó khớp đúng.

Bảng 8: Hiệu quả ReID theo từng kịch bản thực nghiệm

Kịch bản	Hiệu quả	Ghi chú
Che khuất ngắn	Tốt	Tầng jacket logic (vị trí + thời gian) xử lý được
Hai cầu thủ khác đội giao nhau	Trung bình	Tầng lọc theo đội tránh nhầm chéo đội
Cầu thủ cùng đội tranh chấp	Kém	Áo giống nhau → OSNet khó phân biệt → ID switch
Cầu thủ ra/vào khung khi camera lia	Kém	Vị trí nhảy, vượt ràng buộc vật lý

4.6 Đánh giá Homography, Tốc độ và Quãng đường

Biến đổi phối cảnh (Homography động). Quan sát trên `best.avi`, ngay cả khi camera lia mạnh (giá trị Camera movement X lên tới gần -20 pixel/khung hình tại giây thứ 5), bản đồ chiến thuật 2D vẫn hiển thị vị trí cầu thủ hợp lý trên sân. Điều này cho thấy cơ chế ước lượng ma trận H theo từng khung hình hoạt động hiệu quả trong điều kiện camera di động — một ưu điểm so với phương pháp Homography tĩnh.

Tốc độ và quãng đường. Hệ thống tính và hiển thị được tốc độ (km/h) cùng quãng đường (m) cho từng cầu thủ. Tuy nhiên, độ chính xác còn hạn chế: giá trị độ dịch chuyển camera biến thiên không mượt mà mà nhảy đột ngột (ví dụ $0 \rightarrow -19.61 \rightarrow 0$ giữa các khung hình). Khi camera giật mạnh và đột ngột, module bù trừ chuyển động camera không khử hết được phần dịch chuyển này, khiến vị trí cầu thủ bị “nhảy” trong khoảnh khắc và làm **tốc độ, quãng đường bị thổi phồng cục bộ** tại các thời điểm đó. Cần khắc phục bằng cách làm mượt chuỗi chuyển động camera.

4.7 Đánh giá Báo cáo chiến thuật

Bên cạnh các kết quả tích cực về theo dõi cầu thủ và ánh xạ vị trí lên sơ đồ sân bóng, quá trình đánh giá báo cáo chiến thuật cho thấy hệ thống vẫn tồn tại một số bất nhất giữa dữ liệu đầu vào và các thông kê đầu ra. Các hạn chế này chủ yếu xuất phát từ sai số tích lũy của các module phát hiện đối tượng, theo dõi định danh, phân cụm chiến thuật và phân tích quyền kiểm soát bóng.

Sơ đồ chiến thuật chưa ràng buộc luật bóng đá. Thuật toán phân cụm vị trí tự động phát hiện sơ đồ 7-5-6 cho Đội 1 và 5-8-3 cho Đội 2, tương ứng với 18 và 16 cầu thủ trên sân. Kết quả này vi phạm quy tắc cơ bản của bóng đá khi mỗi đội chỉ được phép có tối đa 11 cầu thủ thi đấu. Nguyên nhân là bước phân cụm chiến thuật (K-Means hoặc DBSCAN) chưa được gán các ràng buộc cứng về số lượng cầu thủ hợp lệ, khiến các đối tượng nhiễu hoặc các trường hợp ghost detection được đưa vào quá trình suy luận đội hình và bị gộp thành các vị trí chiến thuật hợp lệ.

Possession cá nhân và possession đội không nhất quán. Tổng số khung hình kiểm soát bóng của tất cả cầu thủ Đội 1 chỉ đạt 142 frames (xấp xỉ 18.9%), trong khi Đội 2 đạt 254 frames (xấp xỉ 33.9%). Tuy nhiên hệ thống lại báo cáo tỷ lệ kiểm soát bóng của hai đội lần lượt là 46% và 54%. Điều này cho thấy có tới 354 frames (47.2%) không được gán cho bất kỳ cầu thủ nào nhưng vẫn được tính vào thống kê possession của đội. Nguyên nhân xuất phát từ việc possession đội và possession cá nhân được tính toán bởi hai pipeline khác nhau (*ball-team proximity* và *ball-player proximity*) mà chưa có cơ chế đồng bộ hóa hoặc kiểm tra tính nhất quán giữa các kết quả.

Phân tích cầu thủ số #155 cho thấy mâu thuẫn nội tại trong dữ liệu chiến thuật. Cầu thủ này chiếm tới 140 trên tổng số 405 frames possession của Đội 2 (xấp xỉ 34.6%), một giá trị bất thường so với phân bố possession tương đối đồng đều của các cầu thủ còn lại. Đồng thời, kết quả phân vùng chiến thuật cho thấy cầu thủ xuất hiện 100% thời gian trong khu vực *Attacking Third* (*Center Channel*). Tuy nhiên Đội 2 được xác định có hướng tấn công *right_to_left*, nghĩa là phần sân tấn công thực tế phải nằm ở phía bên trái. Sự mâu thuẫn này cho thấy module phân vùng sân chưa thực hiện phép đảo chiều tọa độ hoặc chuẩn hóa hệ quy chiếu theo hướng tấn công của từng đội, dẫn tới việc gán nhãn khu vực chiến thuật chưa chính xác.

Tốc độ trung bình của cầu thủ được tính chưa nhất quán. Một số cầu thủ có giá trị *avg_speed* sai lệch đáng kể so với giá trị tính trực tiếp từ quãng đường di chuyển và thời gian thi đấu. Ví dụ, cầu thủ P3 có tốc độ thực tế khoảng 4.6 km/h nhưng hệ thống báo cáo 13.8 km/h, sai lệch gần 3 lần. Tương tự, cầu thủ P499 có tốc độ tính trực tiếp khoảng 10.0 km/h trong khi giá trị báo cáo đạt 32.3 km/h, sai lệch khoảng 3.2 lần. Nguyên nhân có thể xuất phát từ việc tốc độ trung bình chỉ được tính trên các khung hình mà cầu thủ xuất hiện trong quá trình tracking, trong khi các khoảng thời gian mất dấu không được đưa vào phép tính. Điều này làm tập mẫu bị thiên lệch về các giai đoạn cầu thủ đang di chuyển nhanh, dẫn tới việc đánh giá tốc độ trung bình cao hơn thực tế.

5 Chương 5: Kết Luận Và Hướng Cải Tiến

5.1 Điểm mạnh

- **Biến đổi phối cảnh bằng Homography động (điểm mạnh nổi bật nhất):** Camera broadcast luôn pan/zoom, nên Homography tĩnh chỉ đúng tại một thời điểm và sai ở phần còn lại của trận đấu. Hệ thống ước lượng lại ma trận H **theo từng khung hình** từ điểm mốc sân (YOLO-Pose), nên vị trí cầu thủ trên bản đồ 2D luôn bám đúng dù camera di chuyển. Thực nghiệm cho thấy ngay cả khi camera lia mạnh, bản đồ vẫn định vị hợp lý. Ngoài ra RANSAC loại keypoint nhiễu (H ổn định) và cơ chế fallback (dùng H hợp lệ gần nhất) giúp pipeline không gãy.
- **Nội suy quỹ đạo bóng bù cho hạn chế detection:** Quá trình huấn luyện cho thấy mô hình

phát hiện bóng kém (AP chỉ 0.489, bỏ sót $\approx 53\%$ bóng). Thay vì để pipeline gián đoạn, hệ thống dùng **nội suy tuyến tính** để tính lại bounding box bóng ở các khung hình bị bỏ sót, giúp quỹ đạo bóng liên tục — một giải pháp thực dụng được chính kết quả thực nghiệm chứng minh là cần thiết.

- **Một số điểm mạnh khác:** ReID 5 tầng có ràng buộc vật lý (loại ghép nhầm phi vật lý); kiến trúc mô-đun hóa (dễ kiểm thử riêng từng thành phần); cơ chế stub caching (tăng tốc gỡ lỗi, không phải chạy lại YOLO/ReID mỗi lần).

5.2 Hạn chế

- **Hoán đổi định danh (ID switch) còn nhiều:** Trong ≈ 10 giây đầu video, ID đã leo tới hơn 200 trong khi thực tế chỉ ≈ 26 đối tượng; nhiều cầu thủ bị tái định danh liên tục. Nguyên nhân: cầu thủ cùng đội mặc áo giống hệt; che khuất/tranh chấp; camera lia làm vị trí nhảy.
- **Tốc độ chưa chính xác do camera giật đột ngột:** Dù đã có module bù trừ chuyển động camera, giá trị dịch chuyển camera nhảy bất thường ($0 \rightarrow -20 \rightarrow 0$). Khi camera giật mạnh, optical flow không khử hết, vị trí cầu thủ “nhảy” và làm tốc độ/quãng đường bị thổi phồng cục bộ.
- **Các thống kê chiến thuật còn thiếu nhất quán:** Thuật toán phân cụm đội hình chưa ràng buộc số lượng cầu thủ hợp lệ nên có thể sinh ra các sơ đồ phi thực tế. Chỉ số possession đội và possession cá nhân được tính bằng các pipeline khác nhau dẫn tới sai lệch kết quả. Module phân vùng chiến thuật chưa chuẩn hóa theo hướng tấn công của từng đội, trong khi tốc độ trung bình được tính trên các khung hình theo dõi thành công nên có xu hướng bị thổi phồng. Những hạn chế này làm giảm độ tin cậy của báo cáo chiến thuật và cần được cải thiện trong các nghiên cứu tiếp theo.

5.3 Hướng cải tiến

5.3.1 Cải thiện mô hình

- Tăng dữ liệu bóng (bóng bị che, bay nhanh, ở xa) và xử lý mất cân bằng lớp để nâng AP của ball, giảm phụ thuộc nội suy.
- Fine-tune OSNet trên dữ liệu bóng đá, hoặc thay bằng FastReID / BoT-SORT / StrongSORT để giảm ID switch.
- Làm mượt chuỗi chuyển động camera (Kalman / trung bình trượt) để giảm cú giật, tăng độ chính xác tốc độ.

5.3.2 Mở rộng chức năng

- Nhận diện hành động (chuyền, sút, tắc bóng).
- Vẽ đường việt vị dựa trên vị trí hậu vệ cuối cùng.
- Phân biệt thủ môn riêng theo màu áo khác biệt.

5.3.3 Tối ưu triển khai

- Xử lý thời gian thực; triển khai trên GPU / edge device để tăng tốc và giảm chi phí phần cứng.

5.4 Kết Luận

Bảng 9: Mức độ hoàn thành các mục tiêu đề tài

Mục tiêu	Mức độ hoàn thành	Nhận xét
Phát hiện & theo dõi đa đối tượng	Hoàn thành	Phát hiện tốt cầu thủ/trọng tài/bóng ($mAP@0.5 = 0.85$).
Tái định danh cầu thủ (Re-ID)	Hoàn thành một phần	Đã triển khai ReID 5 tầng nhưng ID switch còn nhiều.
Bù trừ chuyển động camera	Hoàn thành một phần	Có hoạt động nhưng còn giật, chưa khử triệt để.
Biến đổi phối cảnh sang bản đồ 2D	Hoàn thành	Homography động + bản đồ chiến thuật hoạt động tốt.
Ước lượng tốc độ & quãng đường	Hoàn thành một phần	Tính được nhưng còn sai số khi camera giật.
Phân đội & kiểm soát bóng	Hoàn thành	Phân hai đội theo màu áo và hiển thị tỷ lệ kiểm soát bóng.
Trực quan hóa (tactical map, heatmap)	Hoàn thành	Sinh bản đồ chiến thuật 2D với vết di chuyển.
Phân tích & xuất báo cáo	Hoàn thành một phần	Xuất thống kê và báo cáo (phụ thuộc module stats/report).

Đề án đã xây dựng thành công hệ thống phân tích trận đấu bóng đá tự động từ video broadcast một camera, tích hợp pipeline gồm phát hiện (YOLOv11s), theo dõi (ByteTrack), tái định danh

(OSNet), biến đổi phối cảnh động, ước lượng tốc độ/quãng đường, phân đội, tính kiểm soát bóng và trực quan hóa bằng bản đồ chiến thuật 2D. Điểm đóng góp nổi bật là cơ chế **Homography động theo từng khung hình** giúp định vị chính xác khi camera liên tục di chuyển, cùng giải pháp **nội suy quỹ đạo bóng** bù cho hạn chế của khâu phát hiện. Hệ thống vẫn còn hạn chế về hoán đổi định danh và sai số tốc độ khi camera giật, là các hướng sẽ được khắc phục bằng việc fine-tune mô hình ReID, làm mượt chuyển động camera và tối ưu cho xử lý thời gian thực.

Tài Liệu Tham Khảo

1. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. CVPR 2016.
2. Zhang, Y., Sun, P., Jiang, Y., et al. (2022). *ByteTrack: Multi-Object Tracking by Associating Every Detection Box*. ECCV 2022.
3. Zhou, K., Yang, Y., Cavallaro, A., & Xiang, T. (2019). *Omni-Scale Feature Learning for Person Re-Identification*. ICCV 2019.
4. Bewley, A., Ge, Z., Ott, L., Ramos, F., & Upcroft, B. (2016). *Simple Online and Realtime Tracking*. ICIIP 2016.
5. Wojke, N., Bewley, A., & Paulus, D. (2017). *Simple Online and Realtime Tracking with a Deep Association Metric*. ICIIP 2017.
6. Ultralytics. (2024). *YOLOv11 Documentation*. <https://docs.ultralytics.com>
7. torchreid: Deep Learning Person Re-Identification in PyTorch. <https://github.com/KaiyangZhou/deep-person-reid>
8. Dalal, N., & Triggs, B. (2005). *Histograms of Oriented Gradients for Human Detection*. CVPR 2005.
9. Viola, P., & Jones, M. (2001). *Rapid Object Detection using a Boosted Cascade of Simple Features*. CVPR 2001.
10. He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. CVPR 2016.
11. Lin, T.-Y., Dollár, P., Girshick, R., et al. (2017). *Feature Pyramid Networks for Object Detection*. CVPR 2017.